МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

А.В. Коржиманов

ЦИФРОВАЯ ОБРАБОТКА СИГНАЛОВ ПРАКТИКУМ

Учебно-методическое пособие

Рекомендовано [орган, рекомендующий издание] для студентов ННГУ,
обучающихся по направлению подготовки [код и название направления]

Нижний Новгород
2026

УДК [указать]
ББК [указать]
К-66

Рецензенты:

[учёная степень, звание]	**[И.О. Фамилия]**
[учёная степень, звание]	**[И.О. Фамилия]**

К-66 Коржиманов А.В. ЦИФРОВАЯ ОБРАБОТКА СИГНАЛОВ. ПРАКТИКУМ: Учебно-методическое пособие. – Нижний Новгород: Нижегородский госуниверситет, 2026. – 87 с.

Пособие содержит задания восьми практических занятий по цифровой обработке сигналов: дискретизация и спектральный анализ, модуляция, свёртка и корреляция, Z-преобразование, случайные сигналы и спектральная плотность мощности, анализ и синтез цифровых фильтров, нелинейная и адаптивная фильтрация, частотно-временной анализ и вейвлет-преобразования. Каждое занятие открывается краткими сведениями, необходимыми для выполнения заданий: формулами, описанием функций библиотек языка Python и перечнем типичных ошибок. Задания выполняются в среде Jupyter или Google Colab и сопровождаются контрольными вопросами.

Для студентов радиофизического факультета ННГУ, изучающих курс «Цифровая обработка сигналов».

Ответственный за выпуск:
[должность] **[И.О. Фамилия]**

УДК [указать]
ББК [указать]
© Национальный исследовательский
Нижегородский государственный
университет им. Н.И. Лобачевского, 2026

Оглавление

Введение **5**

1 Основы цифровой обработки сигналов: дискретизация, спектральный анализ, алиасинг **7**
Краткие сведения 7
Часть 1. Знакомство с инструментарием 11
Часть 2. Спектральный анализ с помощью БПФ 12
Часть 3. Дискретизация и алиасинг 14

2 Амплитудная и фазовая модуляция **17**
Краткие сведения 17
Часть 1. Генерация и визуализация модулированных сигналов 20
Часть 2. Спектральный анализ модулированных сигналов 22
Часть 3. Демодуляция сигналов 23
Часть 4. Устойчивость к шумам 24

3 Свёртка, корреляция и Z-преобразование **28**
Краткие сведения 28
Задания 32

4 Случайные сигналы, шумы и спектральная плотность мощности **39**
Краткие сведения 39
Задания 40

5 Основные характеристики цифровых фильтров **43**
Краткие сведения 43
Подготовка рабочей среды 46
Часть 1. КИХ-фильтр: скользящее среднее 46
Часть 2. БИХ-фильтр: экспоненциальное сглаживание 48
Часть 3. Фильтр Савицкого – Голея (Savitzky–Golay) 50
Часть 4. Сравнение КИХ и БИХ фильтров 52
Часть 5. Нелинейная фильтрация: медианный фильтр 53

6 Синтез цифровых фильтров (КИХ и БИХ) 56
Краткие сведения 56
Часть 1. Синтез КИХ-фильтров методом окон 60
Часть 2. Равноволновой синтез КИХ-фильтров (Паркса–МакКлеллана) 61
Часть 3. Синтез БИХ-фильтров 63
Часть 4. Сравнение КИХ и БИХ фильтров одинакового порядка . . 64

7 Адаптивная фильтрация и частотно-временной анализ 66
Краткие сведения 66
Подготовка рабочей среды 69
Часть 1. Адаптивные фильтры 69
Часть 2. Частотно-временной анализ 72

8 Вейвлет-преобразования 76
Краткие сведения 76
Часть 1. Непрерывное вейвлет-преобразование 79
Часть 2. Дискретное вейвлет-преобразование 80
Часть 3. Применения 82

Список литературы 85

Введение

Практикум составлен на основе практических занятий, сопровождающих курс лекций «Цифровая обработка сигналов», который автор читает студентам радиофизического факультета Нижегородского государственного университета имени Н.И. Лобачевского. Восемь занятий практикума следуют за восемью лекциями курса: от дискретизации сигналов и дискретного преобразования Фурье через свёртку, корреляционный и спектральный анализ к анализу и синтезу цифровых фильтров, адаптивной фильтрации, частотно-временному анализу и вейвлет-преобразованиям.

Цель практикума — научить применять методы цифровой обработки сигналов на практике: генерировать и анализировать сигналы, проектировать фильтры, оценивать спектры, сравнивать между собой различные методы и делать обоснованные выводы о том, какой из них лучше подходит для конкретной задачи. Большинство заданий построено как численный эксперимент: требуется не только получить результат, но и исследовать его зависимость от параметров и объяснить наблюдаемое поведение.

Задания выполняются на языке Python с использованием библиотек NumPy, SciPy и Matplotlib, а в последнем занятии — библиотеки PyWavelets. Для каждого занятия подготовлен ноутбук Jupyter, который можно открыть в сервисе Google Colab, не устанавливая ничего на свой компьютер. Ноутбуки, пояснительные заметки и необходимые аудиофайлы размещены в открытом репозитории `https://github.com/korzhimanov/dsp-seminars`: задания находятся в каталоге `seminars`, заметки — в каталоге `notes`, данные — в каталоге `data`. Порядок работы с репозиторием и Google Colab описан в кратких сведениях к первому занятию.

Каждая глава соответствует одному занятию и состоит из двух частей. Раздел «Краткие сведения» содержит формулы, описание используемых функций библиотек и перечень типичных ошибок; он не заменяет лекций, а служит справочником при выполнении заданий. Далее следуют сами задания, сгруппированные по темам. Фрагменты программного кода, приведённые в условиях, входят в условие задачи: это вспомогательные функции и заготовки, в которых строки с комментарием «Ваш код здесь» отмечают места, требующие доработки.

Результатом выполнения каждого задания являются программный код,

графики и ответы на контрольные вопросы, записанные непосредственно в ноутбуке. Все графики должны иметь подписи осей с единицами измерения, заголовки и легенды. Ответы на вопросы следует давать в виде кратких, но полных выводов, опирающихся на полученные результаты и теорию, изложенную в лекциях.

Автор надеется, что практикум поможет читателям почувствовать, как теоретические понятия цифровой обработки сигналов превращаются в работающие алгоритмы, и научиться выбирать подходящий инструмент для решения реальных задач.

Занятие 1

Основы цифровой обработки сигналов: дискретизация, спектральный анализ, алиасинг

Краткие сведения

1. Работа с GitHub и Google Colab

Форк учебного репозитория

1. Зайдите на страницу репозитория курса (например, `https://github.com/korzhimanov/dsp-seminars`).
2. Нажмите кнопку **Fork** (справа вверху) – создастся ваша копия репозитория.

Создание своей ветки для редактирования

1. Зайдите на страницу скопированного к себе репозитория.
2. Нажмите кнопку **Branches** (над списком файлов) и затем **New branch** (слева вверху), назовите ветку своей фамилией.

Открытие ноутбука в Colab

- Перейдите в свой форк, смените ветку на вашу, откройте нужный `.ipynb` файл.
- В адресной строке замените `github.com` на `colab.research.google.com/github/` – откроется ноутбук в Colab.
- Подключите свой GitHub к Colab (если ещё нет)
- Либо используйте прямой значок “Open in Colab” (если есть).
- Либо зайдите в Colab, нажмите “Открыть ноутбук” в меню (`Ctrl+O`), найдите нужный репоизторий, смените ветку на вашу и откройте нужный файл

Сохранение результатов и создание Pull Request (PR)

После выполнения задания нужно сохранить изменения в своём форке и отправить PR в основной репозиторий.

В Colab сохраните (`Ctrl+S`) ноутбук и заполните описание коммита.

Зайдите на GitHub и создайте Pull Request из вашего форка в основной репозиторий (вкладка “Pull requests”, кнопка “New pull request”).

2. Основные функции NumPy для создания векторов

- `np.linspace(start, stop, num)` – равномерно распределённые точки между start и stop (включая stop).

```
t = np.linspace(0, 1, 1000, endpoint=False)  # 1000 точек от 0 до 1
```

- `np.arange(start, stop, step)` – точки с фиксированным шагом (stop не включается).

```
t = np.arange(0, 1, 0.001)   # шаг 0.001 с
```

3. Генерация сигналов

Синусоида

$$x(t) = A\sin(\omega t + \phi) = A\sin(2\pi f t + \phi)$$

```
f = 5          # частота, Гц
A = 1          # амплитуда
phi = 0        # начальная фаза (рад)
x = A * np.sin(2 * np.pi * f * t + phi)
```

Меандр (прямоугольный сигнал)

Функция `signal.square` из scipy.signal:

```
f_meandr = 2
meandr = signal.square(2 * np.pi * f_meandr * t)
# Можно задать скважность параметром duty (по умолчанию 0.5 - 50%)
```

4. Построение графиков (matplotlib)

Основные команды

- `plt.plot(x, y)` – линейный график.
- `plt.stem(x, y)` – дискретные отсчёты (“палочки”).
- `plt.xlabel()`, `plt.ylabel()`, `plt.title()` – подписи.
- `plt.grid(True)` – сетка.

- `plt.legend()` – легенда (если указана метка в plot).
- `plt.xlim()`, `plt.ylim()` – пределы осей.
- `plt.show()` – отображение. Пример:

```
plt.plot(t, x, label='сигнал')
plt.stem(t[::20], x[::20], linefmt='r-', markerfmt='ro', basefmt=' ')  # каждый 20-й
↪  отсчёт
plt.xlabel('Время (с)')
plt.ylabel('Амплитуда')
plt.title('Синусоида 5 Гц')
plt.legend()
plt.grid(True)
plt.show()
```

5. Спектральный анализ с помощью БПФ (numpy.fft)

Вычисление спектра

```
N = len(x)                    # количество отсчётов
X = np.fft.fft(x)             # комплексный спектр
freq = np.fft.fftfreq(N, d=1/fs)  # частоты (в Гц) для всех отсчётов
```

Амплитудный спектр (для положительных частот)

```
half = N // 2
freq_pos = freq[:half]
amplitude = np.abs(X[:half]) / half   # нормировка
```

Фазовый спектр

```
phase = np.angle(X[:half])      # фаза в радианах
```

Спектр через scipy (альтернатива)

```
from scipy.fftpack import fft, fftfreq   # аналогично numpy
```

6. Интерактивные виджеты (ipywidgets)

Базовый приём – декоратор `@interact` или функция `interact`.

Пример интерактивного графика для демонстрации алиасинга

```
import ipywidgets as widgets

# Функция, которая строит график по значениям амплитуды и частоты
def plot_sine(amplitude, frequency):
    x = np.linspace(0, 2 * np.pi, 200)
    y = amplitude * np.sin(frequency * x)

    plt.figure(figsize=(8, 4))
    plt.plot(x, y, linewidth=2)
```

```
    plt.title(f'Синусоида: амплитуда = {amplitude:.2f}, частота = {frequency:.2f}')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.grid(True)
    plt.ylim(-2, 2)   # фиксируем пределы по Y
    plt.show()

# Создаём два FloatSlider явно
amp_slider = widgets.FloatSlider(value=1.0, min=0.1, max=2.0, step=0.1,
↪  description='Амплитуда:')
freq_slider = widgets.FloatSlider(value=1.0, min=0.5, max=5.0, step=0.1,
↪  description='Частота:')

# Используем interact, передавая слайдеры как аргументы функции
widgets.interact(plot_sine, amplitude=amp_slider, frequency=freq_slider)
```

Другие полезные виджеты

- `IntSlider`, `Dropdown`, `Checkbox` и т.д.

7. Полезные советы

- Для нормировки амплитудного спектра делите модуль БПФ на половину длины (`N//2`), если берёте только положительные частоты.
- Используйте `.copy()` если нужно сохранить исходные данные перед изменением.
- В Colab можно сохранять результаты на свой Google Drive через `drive.mount('/content/drive')`.

Необходимое ПО:

- Python 3 с установленными пакетами `numpy`, `matplotlib`, `scipy`, `ipywidgets` (для интерактивных графиков).
- Jupyter Notebook / Google Colab.

Формат сдачи:

- Один Jupyter Notebook с выполненными заданиями, содержащий код, графики и текстовые ответы на вопросы.
- Имя файла: `1_Фамилия.ipynb`.

Часть 1. Знакомство с инструментарием

Задание 1.1. Импорт библиотек и создание временной оси

Выполните в первой ячейке: Создайте временную ось для сигнала длительностью 1 секунду с частотой дискретизации 1000 Гц (т.е. 1000 отсчётов в секунду). Выведите длину массива времени и шаг дискретизации, проверьте, что они совпадают с ожидаемыми значениями.

```
fs = 1000          # частота дискретизации, Гц
T = 1              # длительность сигнала, с
t = []  # временная ось

print(f"Длина массива времени: {0} отсчётов")
print(f"Шаг дискретизации (период): {0} с")
```

Вопрос: Чему равен период дискретизации T_s? Как связаны частота дискретизации f_s и период T_s?

Задание 1.2. Генерация простых сигналов

Сгенерируйте и визуализируйте:

- Синусоидальный сигнал частотой 5 Гц, амплитудой 1.
- Сумму двух синусоид с частотами 5 и 12 Гц и одинаковыми амплитудами.
- Прямоугольный меандр частотой 2 Гц (используйте `signal.square`).

Для каждого сигнала постройте график временной зависимости (первые 0.5 секунды).

```
# Синусоида 5 Гц
f1 = 5
sin5 = []

# Сумма синусоид 5 и 12 Гц
f2 = 12
sum_sin = []

# Меандр 2 Гц
meander = []

# Визуализация
plt.figure(figsize=(14, 4))

plt.subplot(1, 3, 1)
plt.plot([], [])  # первые 0.5 с
plt.title('Синусоида 5 Гц')
```

```
plt.xlabel('Время (с)')
plt.ylabel('Амплитуда')
plt.grid(True)

plt.subplot(1, 3, 2)
plt.plot([], [])
plt.title('Сумма 5 Гц + 12 Гц')
plt.xlabel('Время (с)')
plt.grid(True)

plt.subplot(1, 3, 3)
plt.plot([], [])
plt.title('Меандр 2 Гц')
plt.xlabel('Время (с)')
plt.grid(True)

plt.tight_layout()
plt.show()
```

Вопрос: Изменится ли вид суммы синусоид, если одну из них сдвинуть по фазе на $90°$ ($\pi/2$), на $180°$ (π), на $360°$ (2π)? (Проверьте на практике, изменив фазу второго сигнала). Объясните результат.

Часть 2. Спектральный анализ с помощью БПФ

Задание 2.1. Спектр синусоиды

Для синусоиды частотой 5 Гц вычислите БПФ (`np.fft.fft`). Постройте амплитудный спектр (модуль БПФ) для положительных частот. Не забудьте правильно нормировать амплитуды (поделить на количество отсчётов и учесть, что мы берём половину спектра). Частотную ось получите с помощью `np.fft.fftfreq`.

Подпишите оси: частота (Гц), амплитуда.

```
# БПФ синусоиды 5 Гц
X = []
freq = []

# Берём только положительные частоты
half = len(t)//2
X_mag = []  # не забываем про нормировку амплитуды
freq_pos = [] # только положительные частоты

plt.figure()
plt.stem([0,], [0,], basefmt=" ")
plt.title('Амплитудный спектр синусоиды 5 Гц')
plt.xlabel('Частота (Гц)')
plt.ylabel('Амплитуда')
plt.xlim(0, 20)
```

```
plt.grid(True)
plt.show()
```

Вопрос: Почему в спектре присутствует только один пик? Какая должна быть его амплитуда теоретически? Совпадает ли с полученной?

Задание 2.2. Спектр суммы синусоид

Для суммы синусоид (5 и 12 Гц) постройте амплитудный спектр. Убедитесь, что видны два пика на ожидаемых частотах. Измерьте их амплитуды.

```
X_sum = []
X_sum_mag = []

plt.figure()
plt.stem([0,], [0,], basefmt=" ")
plt.title('Спектр суммы 5 Гц и 12 Гц')
plt.xlabel('Частота (Гц)')
plt.ylabel('Амплитуда')
plt.xlim(0, 20)
plt.grid(True)
plt.show()
```

Вопрос: Изменится ли амплитудный спектр, если фазы сигналов изменить? Проверьте, задав для одного из них случайную фазу.

Задание 2.3. Спектр меандра

Постройте амплитудный спектр меандра частотой 2 Гц. Сравните с теоретическим представлением: меандр состоит из нечётных гармоник с убывающими амплитудами $(1, 1/3, 1/5, \dots) \cdot 4/\pi$.

```
X_meander = []
X_meander_mag = []

plt.figure()
plt.stem([0,], [0,], basefmt=" ")
plt.title('Спектр меандра 2 Гц')
plt.xlabel('Частота (Гц)')
plt.ylabel('Амплитуда')
plt.xlim(0, 30)
plt.grid(True)
plt.show()
```

Вопрос: Почему в спектре меандра присутствуют только нечётные гармоники? Покажите, что амплитуды гармоник совпадают с теоретическим значением.

Часть 3. Дискретизация и алиасинг

Задание 3.1. Корректная и некорректная дискретизация

Создайте «аналоговый» сигнал — синусоиду частотой 8 Гц, но с очень высокой частотой дискретизации (например, 1000 Гц), чтобы он выглядел гладким.

```
# «Аналоговый» сигнал с высокой частотой дискретизации
fs_analog = 1000
t_analog = []
f_signal = 8
analog_signal = []

plt.figure(figsize=(12,5))
plt.plot([], [], 'b-', alpha=0.3, label='Исходный сигнал (8 Гц)')
plt.xlabel('Время (с)')
plt.ylabel('Амплитуда')
plt.title('Исходный сигнал (8 Гц)')
plt.legend()
plt.grid(True)
plt.xlim(0, 0.5)
plt.show()
```

Теперь выполните его дискретизацию с двумя разными частотами:

- $f_{s1} = 50$ Гц (заведомо больше удвоенной частоты сигнала, 2·8=16 Гц);
- $f_{s2} = 12$ Гц (меньше 16 Гц).

Для этого создайте новые временные оси с шагом $1/f_{s1}$ и $1/f_{s2}$ и вычислите значения синусоиды в этих точках.

На одном графике изобразите:

- исходный гладкий сигнал (тонкая линия);
- отсчёты при корректной дискретизации (маркеры);
- отсчёты при алиасинге (маркеры другого цвета).

Также постройте сигнал, который «воспринимается» после дискретизации с малой частотой.

```
# Частоты дискретизации для эксперимента
fs1 = 50
fs2 = 12

# Отсчёты при корректной дискретизации
t1 = []
samples1 = []

# Отсчёты при некорректной дискретизации (алиасинг)
```

```
t2 = []
samples2 = []

plt.figure(figsize=(12,5))
plt.plot([], [], 'b-', alpha=0.3, label='Исходный сигнал (8 Гц)')
plt.stem([0,], [0,], linefmt='g-', markerfmt='go', basefmt=' ', label=f'Отсчёты
↪  fs={fs1} Гц')
plt.stem([0,], [0,], linefmt='r-', markerfmt='ro', basefmt=' ', label=f'Отсчёты
↪  fs={fs2} Гц')
plt.plot([], [], 'm--', label='Восстановленный сигнал (по fs2)')
plt.xlabel('Время (с)')
plt.ylabel('Амплитуда')
plt.title('Сравнение корректной и некорректной дискретизации')
plt.legend()
plt.grid(True)
plt.xlim(0, 0.5)
plt.show()
```

Вопрос: Какая частота «кажется» присутствующей в сигнале при $f_{s2} = 12$ Гц? Совпадает ли она с предсказанной теоретически? (Подсказка: формула алиасинга $f_{alias} = |f - kf_s|$, выберите k так, чтобы f_{alias} лежала в интервале $[0, f_s/2]$.)

Задание 3.2. Интерактивная демонстрация алиасинга

Используя `ipywidgets.interact`, создайте интерактивный график, где можно менять частоту сигнала и частоту дискретизации ползунками, и сразу видеть исходный сигнал, отсчёты и восстановленный сигнал. Пример такого кода был на лекции.

Добейтесь, чтобы при нарушении условия Найквиста явно проявлялся алиасинг (восстановленный сигнал имел более низкую частоту, чем исходный).

```
def plot_aliasing(f_signal, f_sampling):

    plt.figure(figsize=(12,4))
    plt.plot()
    plt.xlabel('Время (с)')
    plt.ylabel('Амплитуда')
    plt.title(f'Частота сигнала = {f_signal} Гц, fs = {f_sampling} Гц')
    plt.legend()
    plt.grid(True)
    plt.xlim(0, 0.5)
    plt.show()

widgets.interact(plot_aliasing,
         f_signal=widgets.FloatSlider(min=, max=, step=, value=),
         f_sampling=widgets.FloatSlider(min=, max=, step=, value=));
```

Вопрос: При каких соотношениях частоты сигнала и частоты дискретизации алиасинг наиболее заметен? Что происходит, когда частота сигнала в точности равна половине частоты дискретизации?

Занятие 2

Амплитудная и фазовая модуляция

Краткие сведения

1. Амплитудная модуляция (АМ)

Формула АМ-сигнала:

$$s_{AM}(t) = A_c\big[1 + a_m \cdot m(t)\big]\cos(2\pi f_c t)$$

где:

- A_c – амплитуда несущей,
- f_c – частота несущей,
- $m(t)$ – модулирующий сигнал (сообщение), **нормированный** так, чтобы $\max|m(t)| = 1$,
- a_m – глубина (амплитуда) модуляции.

Пример генерации в Python:

```
fs = 10000          # частота дискретизации
t = np.linspace(0, 0.1, int(fs*0.1), endpoint=False)
fc = 1000           # несущая
fm = 100            # частота сообщения
a_m = 0.8           # амплитуда модуляции
m_t = np.sin(2*np.pi*fm*t)   # сообщение
s_am = (1 + a_m*m_t) * np.cos(2*np.pi*fc*t)
```

2. Фазовая модуляция (ФМ)

Формула ФМ-сигнала:

$$s_{PM}(t) = A_c\cos\big(2\pi f_c t + \beta\, m(t)\big)$$

где β – **индекс фазовой модуляции** (максимальное отклонение фазы в радианах).

Пример генерации:

```
beta = 5
s_pm = np.cos(2*np.pi*fc*t + beta * m_t)
```

3. Преобразование Гильберта и демодуляция

Аналитический сигнал

Для демодуляции АМ и ФМ сигналов будем использовать так называемый аналитический сигнал, у которого действительная часть – исходный сигнал, а мнимая – его преобразование Гильберта:

$$\mathcal{H}[x](t) = \frac{1}{\pi}\,\text{p.v.}\int_{-\infty}^{\infty}\frac{x(\tau)}{t-\tau}\,d\tau$$

где p.v. — главное значение интеграла по Коши.

```
from scipy.signal import hilbert
z = hilbert(s)               # аналитический сигнал
```

Аналитический сигнал позволяет извлечь огибающую и мгновенную фазу исходного сигнала:

- **Огибающая** (envelope) – модуль аналитического сигнала: `envelope = np.abs(z)`
- **Мгновенная фаза** – аргумент: `phase = np.angle(z)` (значения от $-\pi$ до π)

Демодуляция АМ

Для АМ огибающая равна $A_c[1 + m(t)]$. Чтобы восстановить $m(t)$:

```
envelope = np.abs(hilbert(s_am))
m_recovered = envelope / Ac - 1   # если Ac=1, то просто envelope - 1
```

Демодуляция ФМ

Мгновенная фаза содержит линейный тренд $2\pi f_c t$ плюс полезное отклонение $\beta m(t)$. Нужно:

1. Получить мгновенную фазу и устранить скачки (unwrap).
2. Вычесть линейную составляющую.
3. Разделить на β.

```
z = hilbert(s_pm)
phase = np.unwrap(np.angle(z))              # развёрнутая фаза
phase_no_carrier = phase - 2*np.pi*fc*t      # убираем тренд несущей
m_recovered = phase_no_carrier / beta
```

Важно: без `np.unwrap` скачки фазы на $\pm\pi$ приведут к ложным выбросам.

4. Функция `np.unwrap` – корректировка фазы

При вычислении фазы через `np.angle` результат лежит в пределах $[-\pi, \pi]$. Если истинная фаза пересекает эти границы, возникают скачки на 2π. `np.unwrap` добавляет или вычитает 2π в местах скачков, восстанавливая непрерывную фазу.

```
phase_wrapped = np.angle(z)
phase_unwrapped = np.unwrap(phase_wrapped)
```

5. Функция `np.std` для сравнения сигналов

Для количественной оценки ошибки между исходным сообщением $m(t)$ и восстановленным $\hat{m}(t)$ используют среднеквадратичное отклонение (СКО, Root Mean Square Error, RMSE):

$$\text{СКО} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - y_i)^2}$$

```
error = np.std(m_original - m_recovered)
print(f'СКО = {error:.4f}')
```

Чем меньше значение СКО, тем точнее восстановление.

6. Работа с WAV-файлами

Чтение файла:

```
from scipy.io import wavfile
rate, data = wavfile.read('filename.wav')
```

- `rate` – частота дискретизации (Гц)
- `data` – массив целых чисел. Форма:
  - моно: `(N,)`
  - стерео: `(N, 2)` (левый и правый каналы) Для монофонической обработки возьмите один канал: `data = data[:, 0]` (если стерео).

Прослушивание в Jupyter/Colab:

```
from IPython.display import Audio
Audio(data_float, rate=rate)
```

7. Амплитудный шум (генерация)

Для моделирования амплитудных помех удобно использовать **мультипликативный шум**: случайные флуктуации амплитуды несущей.

$$y(t) = x(t) \cdot (1 + n(t))$$

где $n(t)$ — шумовой (случайный) процесс, причём $\langle n(t) \rangle = 0$.

Генерация шума с помощью `np.random.randn`:

```
noise = np.random.randn(N)          # возвращает N случайных величины в интервале (0,
↪  1), распределённых по Гауссу
noise_scaled = amplitude_noise * noise   # задаём уровень
s_noisy = s * (1 + noise_scaled)    # мультипликативный шум
```

Здесь `amplitude_noise` – относительный уровень шума (например, 0.2 означает $\pm 20\%$ флуктуации).

8. Полезные советы

- Всегда проверяйте, что модулирующий сигнал находится в диапазоне [-1, 1].
- При использовании `hilbert` надо помнить о краевых эффектах – лучше брать сигнал достаточной длины и отбрасывать края при оценке точности.
- Для построения спектров можно использовать `np.fft.rfft` (односторонний спектр)
- При сравнении сигналов и их спектров не забывайте про нормировку.

Часть 1. Генерация и визуализация модулированных сигналов

Импорт библиотек и параметры

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from scipy.io import wavfile
from scipy.signal import hilbert
import ipywidgets as widgets
from IPython.display import Audio, display

%matplotlib inline
plt.rcParams['figure.figsize'] = (12, 5)

# Параметры дискретизации для демонстрации
fs = 10000                      # частота дискретизации (Гц)
```

```
T = 0.1                          # длительность сигнала (с)
t = np.linspace(0, T, int(fs*T), endpoint=False)

# Несущая
fc = 1000                        # частота несущей (Гц)
Ac = 1.0                         # амплитуда несущей
```

Модулирующий сигнал

В качестве модулирующего сигнала используйте сумму двух синусоид с частотами 100 Гц и 150 Гц, нормализованную к диапазону [-1, 1]. Постройте его график.

```
fm1 = 100
fm2 = 150
m_t = [] # модулирующий сигнал - сумма двух синусоид со значениями в интервале
↪  [-1,1]

plt.figure()
plt.plot(t, m_t)
plt.title('Модулирующий сигнал (100 Гц + 200 Гц)')
plt.xlabel('Время (с)')
plt.ylabel('Амплитуда')
plt.grid(True)
plt.show()
```

Амплитудная модуляция (АМ)

Формула АМ:

$$s_{AM}(t) = A_c[1 + a_{AM} \cdot m(t)] \cos(2\pi f_c t)$$

Параметр a_{AM} регулирует *глубину модуляции*, он не может быть больше 1 (при $a_{AM} > 1$ и исходном сигнале $m(t)$, нормализованном на единицу, амплитуда результирующего сигнала в некоторые промежутки времени становится отрицательной).

```
a_am = 0.2
s_am = []
```

Фазовая модуляция (ФМ)

Формула ФМ:

$$s_{PM}(t) = A_c \cos(2\pi f_c t + \beta \cdot m(t)),$$

где β – индекс фазовой модуляции.

```
beta_pm = 5
s_pm = []
```

Задание 1.1

Постройте на одном графике модулирующий сигнал и оба модулированных сигнала (АМ, ФМ). Оси и графики должны быть подписаны.

Часть 2. Спектральный анализ модулированных сигналов

Задание 2.1. Функция построения амплитудного спектра

Реализуйте функцию построения амплитудного спектра `plot_spectrum(s,fs,title)` Значения аргументов:

- `s` - массив, содержащий значения сигнала
- `fs` - частота отсчётов сигнала
- `title` - заголовок графика, выводимый сверху

На графике по горизонтальной оси должна быть отложена частота в герцах, а по вертикальной - амплитуда гармоники. График должен строиться только для положительных значений частоты. Оси должны быть подписаны.

```
def plot_spectrum(s, fs, title):
  # ваш код здесь
```

Спектр АМ

Постройте спектр амплитудно-модулированного сигнала

```
plot_spectrum(s_am, fs, 'AM спектр (a_am = {a_am} по глубине)')
```

Задание 2.2. Спектр ФМ при разных индексах модуляции

Постройте спектр фазово-модулированного сигнала

```
plot_spectrum(s_pm, fs, f'ФМ спектр (β = {beta_pm})')
```

Постройте на одном графике спектры для разных индексов модуляции (например, $\beta = 0.1, 0.3, 1, 2, 5, 10$). Оси и графики должны быть подписаны.

Задание 2.3. Интерактивное исследование спектра ФМ

Создайте интерактивный график для исследования зависимости спектра ФМ-сигнала от индекса модуляции.

```
def pm_spectrum(beta=5):
  # ваш код здесь

widgets.interact(pm_spectrum, beta=widgets.FloatSlider(min=0.1, max=20, step=0.1,
↪  value=5));
```

Вопросы:

- Почему спектр АМ содержит только пять ярко выраженных компоненты (несущая и по две боковых с каждой стороны от неё)?
- Опишите, как выглядит спектр ФМ при малых β?
- Что происходит с амплитудой несущей и боковыми полосами ФМ при увеличении β?

Часть 3. Демодуляция сигналов

Задание 3.1. Демодуляция АМ с помощью преобразования Гильберта

Преобразование Гильберта позволяет получить аналитический сигнал, огибающая которого содержит исходное сообщение.

```
# Аналитический сигнал для АМ
analytic_am = hilbert(s_am)
envelope = np.abs(analytic_am) # огибающая = 1 + m(t)

# Убираем постоянную составляющую
demod_am = envelope - 1

# Нормализуем сигнал
demod_am = demod_am/a_am

# Сравнение с исходным
plt.figure()
plt.plot(t, m_t, label='Исходное сообщение')
plt.plot(t, demod_am, '--', label='Демодулированное (АМ)')
plt.xlabel('Время (с)')
plt.ylabel('Амплитуда')
plt.title('Демодуляция АМ сигнала')
plt.legend()
plt.grid(True)
plt.show()
```

Вычислите среднеквадратичную ошибку между исходным и восстановленным сообщением с помощью функции `np.std`.

Демодуляция ФМ сигнала

Для ФМ информация заключена в отклонении фазы от линейного тренда. Используем аналитический сигнал и выделим мгновенную фазу.

```
# Аналитический сигнал для ФМ
analytic_pm = hilbert(s_pm)
inst_phase = np.unwrap(np.angle(analytic_pm))   # развернутая фаза

# Убираем линейную составляющую (фаза несущей)
phase_no_carrier = inst_phase - 2 * np.pi * fc * t

# Убираем постоянную составляющую (может появиться в общем случае при наличии
↪  шумов, искажений и т. п.)
demod_pm = phase_no_carrier - np.mean(phase_no_carrier)

# Нормализуем сигнал
demod_pm = demod_pm / beta_pm

# Сравнение с исходным
plt.figure()
plt.plot(t, m_t, label='Исходное сообщение')
plt.plot(t, demod_pm, '--', label='Демодулированное (ФМ)')
plt.xlabel('Время (с)')
plt.ylabel('Амплитуда')
plt.title('Демодуляция ФМ сигнала')
plt.legend()
plt.grid(True)
plt.show()
```

Вопрос: Почему важно использовать `np.unwrap` для фазы?

Часть 4. Устойчивость к шумам

В этой части мы работаем с реальным аудиофайлом. Для примера WAV-файл расположен в том же репозитории, что и ноутбук по адресу `data/sample-3s.wav`, но его надо загрузить в Google Colab.

Загрузка аудио

```
# Загружаем файл и сохраняем под именем audio.wav
!wget -O audio.wav "https://github.com/korzhimanov/dsp-seminars/raw/refs/heads/mai
↪  n/data/sample-3s.wav"
```

```
fs_audio, melody_stereo = wavfile.read('audio.wav') # читаем wav-файл в массив
↪  numpy, в файл записан стереосигнал, поэтому melody_stereo будет содержать две
↪  дорожки (левую и правую)

melody = melody_stereo[:,0] # переходим к моносигналу, беря только одну из двух
↪  дорожек
melody = melody / np.max(np.abs(melody)) # нормализуем сигнал, чтобы корректно
↪  работала амплитудная модуляция

t_audio = np.linspace(0, melody.shape[0]/fs_audio, melody.shape[0], endpoint=False)
↪  # для визуализации создаём также массив временных отсчётов

print(f"Частота дискретизации: {fs_audio} Гц")
print(f"Длина временного ряда: {len(melody)}")
print(f"Тип данных: {melody.dtype}")

print("Исходное аудио:")
display(Audio(melody, rate=fs_audio))
```

Задание 4.1. Осциллограмма и спектр сигнала

Постройте графики самого сигнала и его спектра. Подпишите оси. **Вопрос:** До каких частот наблюдаются заметные пики в сигнале? Подумайте, какую частоту несущей можно выбрать при частоте дискретизации 44,1 кГц, чтобы эти пики не потерять. Как на это влияет выбор индекса модуляции ФМ-сигнала?

Задание 4.2. Подготовка к модуляции (интерполяция)

Выберите частоту несущей в соответствии с ответом на предыдущий вопрос (если восстановленный сигнал будет сильно зашумлён, попробуйте другие значения).

```
fc_high = 5000 # несущая 5 кГц
Ac = 1
```

Модуляция аудиосигнала

Сгенерируйте модулированные сигналы. Индекс модуляции для ФМ-сигнала выберите так, чтобы несильно портить модулируемый сигнал.

```
# AM
a_am = 0.2
s_am_audio = Ac * (1 + a_am * melody) * np.cos(2 * np.pi * fc_high * t_audio)

# ФМ
```

```
beta_pm_audio = 2
s_pm_audio = Ac * np.cos(2 * np.pi * fc_high * t_audio + beta_pm_audio * melody)
```

Добавление шумов

Амплитудный шум: мультипликативный гауссов шум.

```
noise_amp = 0.1 * np.random.randn(len(t_audio))
s_am_noisy = s_am_audio * (1 + noise_amp)
s_pm_noisy = s_pm_audio * (1 + noise_amp)
```

Задание 4.3. Демодуляция сигналов

Демодуляция АМ:

Используйте преобразование Гильберта, чтобы демодулировать АМ-сигнал без шума, убедитесь, что качество звука изменилось незначительно. Демодулируйте зашумлёный АМ-сигнал. **Вопросы:**

- Изменилось ли качество звука?
- Зафиксируйте $a_{AM} = 0{,}2$. Попробуйте поменять амплитуду шума. При каких значениях амплитуды вы слышите заметную разницу с оригиналом?
- Зафиксируйте амплитуду шума равную 0,01. Попробуйте поменять a_{AM}, при каких амплитудах модуляции начинаются заметные искажения? Сделайте вывод о соотношении a_{AM} и допустимых уровнях шума. **Демодуляция ФМ:**

Используйте метод аналитического сигнала для демодуляции ФМ-сигнала. Убедитесь, что качество звука изменилось незначительно. Демодулируйте зашумлёный ФМ-сигнал. **Вопросы:**

- Изменилось ли качество звука?
- Зафиксируйте $\beta = 2$. Попробуйте поменять амплитуду шума. При каких значениях амплитуды вы слышите заметную разницу с оригиналом?
- Зафиксируйте амплитуду шума равную 0,1. Попробуйте поменять β. Как субъективная величина искажений зависит от β?

Задание 4.4. Сравнение спектров

Выберите значение амплитуды шума и величин a_{AM} и β так, чтобы искажения были заметны, но не велики. Постройте спектры исходного сигнала и сигналов, восстановлённых из зашумлённых АМ и ФМ-сигналов. **Вопрос:**

- Заметны ли визуально искажения спектров для случаев АМ и ФМ? Постройте спектры АМ и ФМ сигналов. Зафиксируйте $\beta = 2$, подберите a_{AM} так, чтобы амплитуды гармоник в обоих случаях были сравнимы по величине. **Вопрос:**
- При какой амплитуде шума в этом случае вы слышите заметные искажения АМ сигнала? А ФМ сигнала? Сделайте вывод.

Занятие 3

Свёртка, корреляция и Z-преобразование

Краткие сведения

1. Функции Python для свёртки и корреляции

`np.convolve(x, h, mode)`

- `mode='full'` – полная линейная свёртка (длина $N_x + N_h - 1$).
- `mode='same'` – центральная часть той же длины, что и x.
- `mode='valid'` – только те точки, где ядро полностью перекрывается с сигналом.

`np.correlate(x, y, mode)`

Аналогичные режимы. При `mode='full'` результат имеет длину $N_x + N_y - 1$. Индекс 0 соответствует полному перекрытию сигналов.

Для получения сдвига, при котором достигается максимум корреляции, используется формула:

```
corr = np.correlate(x, y, mode='full')
delay = np.argmax(corr) - (len(x) - 1)
```

2. Гауссовы окна и их свёртка

Гауссово окно можно сгенерировать с помощью `scipy.signal.windows.gaussian(M, std)`.

Аналитическое свойство: свёртка двух гауссовых функций с дисперсиями σ_1^2 и σ_2^2 даёт гауссову функцию с дисперсией $\sigma_1^2 + \sigma_2^2$:

$$(f_{\sigma_1} * f_{\sigma_2})(t) = f_{\sqrt{\sigma_1^2+\sigma_2^2}}(t).$$

Это свойство используется в задании 1 для сравнения численного результата с теоретическим.

3. Фильтрация с помощью свёртки

Прямоугольное окно (усреднение):

```
h_rect = signal.windows.boxcar(L)/L
```

Простое скользящее среднее.

Гауссовское окно:

```
h_gauss = signal.windows.gaussian(L, std)
h_gauss = h_gauss / np.sum(h_gauss)  # нормализация
```

4. Поиск временной задержки с помощью кросс-корреляции

Идея: если $y[n] = x[n - d] +$ шум, то кросс-корреляция $R_{xy}[m]$ имеет максимум при $m = d$.

Алгоритм:

1. Вычислить `corr = np.correlate(x, y, mode='full')`.
2. Найти индекс максимума: `idx = np.argmax(corr)`.
3. Задержка = `idx - (len(x)-1)`.

5. Обнаружение шаблона в зашумлённом сигнале

Задача: найти в длинном сигнале позицию, где содержится известный короткий шаблон.

Метод:

- Вычислить кросс-корреляцию между длинным сигналом и шаблоном (удобно использовать `mode='valid'`, чтобы избежать краевых эффектов).
- Пик корреляции указывает на позицию совпадения.

Согласованный фильтр. Для обнаружения известного сигнала $s[n]$ в белом шуме с дисперсией σ^2 оптимален фильтр с импульсной характеристикой $h[n] = s[n_0 - n]$ – это и есть корреляция с шаблоном. Максимальное отношение сигнал/шум на выходе равно E_s/σ^2, где $E_s = \sum s^2[n]$. Если разделить результат корреляции на $\sigma\sqrt{E_s}$, шум на выходе будет иметь единичную дисперсию, а пик – высоту $A\sqrt{E_s}/\sigma$.

```
corr = np.correlate(long_signal, template, mode='valid') /
↪  np.sqrt(np.sum(template**2))
```

6. Работа с аудио в Python

Чтение WAV-файла:

```
from scipy.io import wavfile
rate, data = wavfile.read('file.wav')
```

- `rate` – частота дискретизации (Гц).
- `data` – массив целых чисел. Для стерео форма `(N, 2)`.

Нормализация в диапазон [-1, 1] (для 16-битного аудио):

```
data = data / 32767.0
```

Прослушивание в Jupyter/Colab:

```
from IPython.display import Audio
Audio(data, rate=rate)
```

Поиск фрагмента:

- Вычислить кросс-корреляцию между полным сигналом и фрагментом.
- Найти индекс максимума.
- Вырезать соответствующий участок и убедиться в совпадении (визуально и на слух).

7. Нормированная корреляция

Коэффициент корреляции не зависит от амплитуд сигналов и лежит в диапазоне $[-1, 1]$:

$$\rho_{xy}[m] = \frac{R_{xy}[m]}{\sqrt{R_{xx}[0]\, R_{yy}[0]}}.$$

При поиске короткого фрагмента s длины L в длинной записи x нормируют на энергию **текущего участка** записи:

$$\rho[m] = \frac{\sum_{n=0}^{L-1} s[n]\, x[n+m]}{\sqrt{E_s \cdot E_x[m]}}, \qquad E_x[m] = \sum_{n=0}^{L-1} x^2[n+m].$$

Энергию всех участков удобно вычислить одной свёрткой:

```
L = len(s)
corr = signal.correlate(x, s, mode='valid', method='fft')   # быстро даже для длинных
↪  записей
c = np.cumsum(np.concatenate([[0], x**2]))
E_x = c[L:] - c[:-L]                                        # энергия каждого участка
↪  длины L
rho = corr / np.sqrt(np.sum(s**2) * E_x + 1e-12)
```

Для длинных аудиозаписей `np.correlate` работает очень медленно (число операций пропорционально произведению длин); `signal.correlate(..., method='fft')` использует БПФ. Без нормировки громкие участки записи дают большие значения корреляции даже без совпадения формы.

8. Z-преобразование в Python

Разностному уравнению $y[n] = \sum_k b_k x[n-k] - \sum_k a_k y[n-k]$ соответствует

$$H(z) = \frac{b_0 + b_1 z^{-1} + \ldots}{1 + a_1 z^{-1} + \ldots}.$$

В `scipy.signal` коэффициенты задаются массивами `b = [b0, b1, ...]` и `a = [1, a1, ...]`. **Знак:** уравнению $y[n] = x[n] + 0.8\, y[n-1]$ соответствует `a = [1, -0.8]`.

- Полюса и нули: `z, p, k = signal.tf2zpk(b, a)`. Если длины `b` и `a` различаются, дополните более короткий массив нулями справа, иначе нули (или полюса) в точке $z = 0$ потеряются.
- Устойчивость: `np.all(np.abs(p) < 1)`.
- Импульсная характеристика: `signal.lfilter(b, a, impulse)`.
- АЧХ: `w, H = signal.freqz(b, a, fs=fs)` – значения $H(z)$ на единичной окружности $z = e^{j2\pi f/f_s}$.
- Полюс $p = re^{j\theta}$ даёт пик АЧХ на частоте $f = \theta f_s/(2\pi)$; чем ближе r к 1, тем пик уже, а импульсная характеристика затухает медленнее (как r^n). Нуль на единичной окружности даёт полное подавление соответствующей частоты.
- Свёртка последовательностей соответствует произведению многочленов: `np.convolve(b1, b2)` = `np.polymul(b1, b2)`.

9. Рекомендации по выполнению заданий

- Для воспроизводимости результатов используйте `np.random.seed(42)` при генерации случайных чисел.
- Все графики должны быть подписаны (оси, заголовки, легенды).
- При анализе влияния шума на точность выполняйте несколько прогонов с разным `np.random.seed()` и усредняйте результаты.
- Для визуализации корреляции полезно показывать и сам сигнал, и корреляционную функцию.

10. Типичные ошибки и их избегание

1. **Путаница режимов `np.convolve`:** всегда проверяйте длину результата.
2. **Краевые эффекты:** при `mode='full'` корреляция даёт значения для всех возможных сдвигов, включая те, где перекрытие сигналов мало. Для поиска задержки это нормально, но при интерпретации нужно учитывать.

3. **Неправильное вычисление сдвига:** используйте `delay = argmax(corr) - (len(x)-1)`.
4. **Ненормализованное аудио:** перед прослушиванием убедитесь, что значения находятся в диапазоне [-1, 1].
5. **Сравнение с аналитикой:** при свёртке гауссовых функций не забывайте нормировать окна (сумма коэффициентов = 1), чтобы амплитуды соответствовали теоретическим.
6. **Знак коэффициентов `a`:** в `scipy.signal` уравнение записывается как $\sum a_k y[n-k] = \sum b_k x[n-k]$, поэтому коэффициенты обратной связи входят в `a` с противоположным знаком.

Задания

Задание 1. Свёртка гауссовых функций: сравнение численного и аналитического результатов

Теоретическое введение

Свёртка двух гауссовых функций даёт гауссову функцию с дисперсией, равной сумме дисперсий. Если

$$f(t) = \frac{1}{\sqrt{2\pi\sigma_1^2}} e^{-t^2/(2\sigma_1^2)}, \quad g(t) = \frac{1}{\sqrt{2\pi\sigma_2^2}} e^{-t^2/(2\sigma_2^2)},$$

то

$$(f * g)(t) = \frac{1}{\sqrt{2\pi(\sigma_1^2 + \sigma_2^2)}} e^{-t^2/(2(\sigma_1^2+\sigma_2^2))}.$$

Задание

1. Сгенерируйте дискретные гауссовы импульсы (например, с помощью `scipy.signal.windows.gaussian`) с заданными стандартными отклонениями `sigma1=3`, `sigma2=5`. Используйте длину окна, достаточную для захвата всей значимой части (например, 10*sigma).
2. Вычислите свёртку численно с помощью `np.convolve`.
3. Постройте графики:
   - Исходные функции.
   - Результат свёртки (численный).
   - Теоретическую гауссову функцию с дисперсией `sigma1^2 + sigma2^2`.
4. Оцените среднеквадратичную ошибку между численным и теоретическим результатами. **Вопросы:**

- Что произойдёт с формой свёртки при увеличении σ?

Задание 2. Фильтрация с помощью свёртки: сравнение прямоугольного и гауссовского окон

Цель

Изучить, как выбор ядра и его длина влияют на подавление высокочастотной составляющей и сохранение низкочастотной.

Задание

1. Создайте сигнал длительностью 2 секунды, частота дискретизации 1000 Гц, состоящий из суммы двух синусоид:

   - низкая частота $f_1 = 5$ Гц, амплитуда 1,
   - высокая частота $f_2 = 80$ Гц, амплитуда 0.5.

2. Сгенерируйте прямоугольное окно длины `L` (например, `L = 21`, используйте `scipy.signal.windows.boxcar(L)`), нормализованное так, чтобы сумма коэффициентов была 1.
3. Сгенерируйте гауссовское окно той же длины (используйте `signal.windows.gaussian(L, std=L/5)`), также нормализованное.
4. Примените свёртку с этими окнами (используйте `mode='same'`).
5. Постройте графики:

   - Исходный сигнал (первые 0.5 с).
   - Отфильтрованные сигналы для обоих окон.

6. Вычислите и сравните амплитуды полезной составляющей (5 Гц) и подавленной (80 Гц) после фильтрации. Для этого:

   - Возьмите БПФ сигналов,
   - Измерьте амплитуды на соответствующих частотах.

7. Исследуйте влияние длины окна: повторите для длин `L = 11, 21, 41, 81`. **Вопросы:**

- Какое окно (прямоугольное или гауссовское) даёт лучшее подавление высокой частоты при одинаковой длине?
- При какой длине прямоугольного и гауссовского окон амплитуда высокочастотного амплитуда падает в 10 раз?
- Как увеличение длины окна влияет на подавление сохранение амплитуды низкой частоты? При какой длине окон она уменьшается более чем на 10%?

Задание 3. Поиск временной задержки с помощью кросс-корреляции

Цель

Определить временной сдвиг между двумя сигналами в присутствии шума и оценить влияние уровня шума на точность.

Задание

1. Сгенерируйте сигнал длительностью 1 секунду (fs=1000 Гц), представляющий собой сумму 100 синусоид со случайными частотами (в диапазоне 10–100 Гц), амплитудами (в диапазоне 0.5–1.5 Гц) и фазами (в диапазоне $0–2\pi$). Воспользуйтесь функцией `numpy.random.uniform`. Нормализуйте полученный сигнал в интервале $[-1; 1]$. Назовём его `x`.
2. Создайте второй сигнал `y`, который является сдвинутой во времени копией `x` на `delay` отсчётов (выберите задержку, например, 100 отсчётов) и добавьте к нему случайный сигнал (шум) с уровнем `noise_level` (например, 0.1) с помощью функции `numpy.random.randn`.
3. Вычислите кросс-корреляцию `corr = np.correlate(x, y, mode='full')` и найдите индекс максимума. Оцените задержку как `delay_est = argmax(corr) - (len(x)-1)` (поскольку `mode='full'` даёт диапазон от -N+1 до N-1).
4. Повторите эксперимент для различных уровней шума в диапазоне от `0` до `2`.
5. Для двух уровней шума (0.3 и 2) постройте график кросс-корреляции и отметьте положение истинной и найденной задержки. **Вопросы:**

- Как зависит точность оценки задержки от уровня шума? При каком уровне шума вычисленная задержка отличается от истинной более чем на 10%? При каком уровне шума вычисленная задержка становится практически неотличима от `0`?
- Почему при высоком уровне шума могут появляться ложные пики?

Задание 4. Обнаружение шаблона в зашумлённом сигнале

Цель

Найти местоположение сигнала, имеющего форму гауссова импульса, модулированного синусоидой, в смеси с шумом. Исследовать влияние отношения сигнал/шум на точность обнаружения.

Задание

1. Создайте шаблон `template` – произведение гауссовой огибающей (σ=100 отсчётов) на синусоиду частотой 20 Гц (при fs=1000 Гц). Длина шаблона примерно 6σ.
2. Создайте длинный сигнал `long_signal` длиной 2000 отсчётов, состоящий из:
   - случайного сигнала (шума), созданного с помощью `numpy.random.randn()` с единичной амплитудой,
   - вставленного в случайную позицию шаблона, умноженного на амплитуду `A` (например, 2).
3. Используйте кросс-корреляцию `np.correlate(long_signal, template, mode='valid')` для поиска позиции шаблона. Найдите индекс максимума корреляции.
4. Повторите эксперимент для разных отношений сигнал/шум (SNR), варьируя амплитуду шаблона от 0.2 до 5.
5. Для одного значения SNR (например, 1) визуализируйте: исходный длинный сигнал, шаблон, результат кросс-корреляции с отмеченным пиком.
6. Корреляция с шаблоном – это **согласованный фильтр**. Разделите результат корреляции на $\sqrt{E_s}$, где $E_s = \sum s^2[n]$ – энергия шаблона: тогда шум на выходе имеет единичную дисперсию. Сравните высоту пика с теоретическим значением $A\sqrt{E_s}$.
7. Замените шаблон гауссовым импульсом **без** модуляции с той же энергией и повторите п. 4. Изменилась ли надёжность обнаружения? **Вопросы:**

- При каком SNR результаты измерения резко ухудшаются?
- Совпадает ли высота пика нормированной корреляции с теоретической $A\sqrt{E_s}$? Почему надёжность обнаружения определяется энергией шаблона, а не его формой?

Задание 5. Поиск фрагмента в реальном аудиосигнале

Цель

Применить кросс-корреляцию для нахождения заданного фрагмента в аудиофайле. Фрагмент и основной файл предоставлены (студентам нужно будет загрузить их).

Задание

1. Загрузите аудиофайл `full_audio.wav` и фрагмент `fragment.wav`. Используйте `scipy.io.wavfile.read`.

2. Вычислите кросс-корреляцию между полным сигналом и фрагментом.
3. Найдите позицию максимального значения корреляции и определите временное смещение (в отсчётах и в секундах).
4. Постройте график кросс-корреляции и отметьте найденный пик.
5. Вырежьте из полного сигнала участок, соответствующий найденной позиции, и прослушайте его (используйте `Audio`). Убедитесь, что он совпадает с фрагментом.
6. Вычислите **нормированную** корреляцию: разделите результат на $\sqrt{E_s \cdot E_x[m]}$, где $E_x[m]$ – энергия участка полного сигнала, с которым сравнивается фрагмент при сдвиге m (см. заметки к занятию). Сравните графики нормированной и ненормированной корреляции.

```
# Загружаем файл и сохраняем под именем audio.wav
!wget -O full_audio.wav "https://github.com/korzhimanov/dsp-seminars/raw/refs/head
↪ s/main/data/full_audio.wav"
!wget -O fragment.wav "https://github.com/korzhimanov/dsp-seminars/raw/refs/heads/
↪ main/data/fragment.wav"
```

Вопросы:

- Почему перед вычислением корреляции сигналы следует нормализовать? Какие ложные максимумы исчезают после нормировки на энергию текущего участка?
- Сколько раз в сигнале встречается вырезанное во фрагменте слово? Удаётся ли с помощью корреляционного анализа определить его во всех случаях? Возникают ли ложные максимумы?
- Что будет, если фрагмент не содержится в полном сигнале? Как это отразится на корреляции?

Задание 6. Z-преобразование: передаточная функция, полюса и нули

Цель

Научиться находить передаточную функцию по разностному уравнению, определять устойчивость системы по расположению полюсов и связывать полюса и нули с видом АЧХ.

Задание

Рассмотрите системы (частота дискретизации $f_s = 1000$ Гц):

- (а) $y[n] = x[n] + 0.8\, y[n-1]$;
- (б) $y[n] = x[n] - 1.1\, y[n-1]$;

- (в) $y[n] = x[n] - x[n-2]$;
- (г) резонатор $y[n] = x[n] + 2r\cos\theta\, y[n-1] - r^2\, y[n-2]$ с $r = 0.95$, $\theta = \pi/5$.

1. Запишите (в ячейке ответа) передаточные функции $H(z)$ и найдите вручную их полюса и нули.
2. Задайте коэффициенты `b`, `a` и найдите полюса и нули с помощью `signal.tf2zpk(b, a)`. Сравните с ручным расчётом. Изобразите их на комплексной плоскости вместе с единичной окружностью (функция `zplane` ниже).
3. Вычислите импульсные характеристики (50 отсчётов, `signal.lfilter`). Какие системы устойчивы? Согласуется ли это с положением полюсов?
4. Для устойчивых систем постройте АЧХ (`signal.freqz(b, a, fs=1000)`). Объясните положение максимумов и провалов АЧХ через положение полюсов и нулей.
5. Для резонатора постройте импульсные характеристики и АЧХ при $r = 0.5,\ 0.9,\ 0.99$. Как ширина резонансного пика и скорость затухания связаны с расстоянием от полюса до единичной окружности?
6. Проверьте теорему о свёртке для Z-преобразования: возьмите два КИХ-фильтра, например `b1 = [1, 1]` и `b2 = [1, -2, 1]`. Убедитесь, что `np.convolve(b1, b2)` совпадает с `np.polymul(b1, b2)`, а АЧХ каскада равна произведению АЧХ отдельных фильтров.

```
def zplane(b, a, ax=None, title=''):
    """Изображает нули (o) и полюса (x) передаточной функции H(z) = B(z)/A(z)."""
    n = max(len(b), len(a))
    b = np.concatenate([b, np.zeros(n - len(b))])   # выравнивание длин, чтобы
    a = np.concatenate([a, np.zeros(n - len(a))])   # не потерять нули и полюса в z
    ↪  = 0
    z, p, k = signal.tf2zpk(b, a)
    if ax is None:
        ax = plt.gca()
    circle = np.exp(1j*np.linspace(0, 2*np.pi, 400))
    ax.plot(circle.real, circle.imag, 'k--', lw=0.8)
    ax.plot(z.real, z.imag, 'o', mfc='none', ms=10, label='нули')
    ax.plot(p.real, p.imag, 'x', ms=10, mew=2, label='полюса')
    ax.set_aspect('equal'); ax.grid(alpha=0.3); ax.legend(); ax.set_title(title)
    return z, p

# (а) y[n] = x[n] + 0.8 y[n-1]
b_a, a_a = [1], [1, -0.8]
zplane(b_a, a_a, title='(а)')
plt.show()

# Ваш код здесь: коэффициенты систем (б), (в), (г); полюса и нули, импульсные
↪  характеристики, АЧХ
```

Вопросы:

- Какие из систем (а)–(г) устойчивы? Сформулируйте условие устойчивости через полюса.
- Почему система (в) пропускает средние частоты и не пропускает постоянную составляющую и частоту $f_s/2$?
- Как угол полюса резонатора θ связан с частотой максимума АЧХ? Чему равна эта частота при $f_s = 1000$ Гц?

Занятие 4

Случайные сигналы, шумы и спектральная плотность мощности

Краткие сведения

1. Генерация белого, розового и красного шума

1. **Гауссовый белый шум:** `np.random.normal(0, scale, N)`.
2. **Розовый шум:**
   - Выполнить БПФ белого шума (`np.fft.rfft`).
   - Создать массив частот `freqs = np.fft.rfftfreq(N, d=1/fs)`.
   - Домножить спектр на `1 / np.sqrt(freqs)`. **Важно:** избегать деления на ноль на нулевой частоте (установить множитель = 0 или 1).
   - Выполнить обратное БПФ (`np.fft.irfft`).
   - Нормализовать до единичной дисперсии (`signal / np.std(signal)`).
3. **Красный шум:** `np.cumsum(white_noise)` и затем нормировка до единичной дисперси.

2. Генерация шума с разной дисперсией

- Для белого шума: `scale=np.sqrt(disp)`.
- Для красного и розового шума после генерации умножить на `np.sqrt(disp)`.

3. Оценка спектральной плотности мощности (СПМ)

- Вычисление периодограммы: `f_per, Pxx_per = scipy.signal.periodogram(signal, fs)`

- Вычисление СПМ методом Уэлча: `f_welch, Pxx_welch = scipy.signal.welch(signal, fs, nperseg=256, noverlap=128, window='hann')`
- Построение графиков в двойном логарифмическом масштабе: `plt.loglog(f, Pxx)`: если $y = x^p$, то $\lg y = p \lg x$.

4. Отношение сигнал-шум (SNR)

Расчет амплитуды шума по SNR:

- Мощность синусоиды: `P_sig = A^2 / 2`.
- SNR (линейный) = `10^(SNR_dB/10)`.
- Мощность шума: `P_noise = P_sig / SNR_lin`.
- Дисперсия шума = `P_noise`.
- `noise = np.random.normal(0, np.sqrt(P_noise), N)`.

Задания

Задание 1. Генерация белого, розового и красного шума

Сгенерируйте три сигнала длительностью 2 секунды с частотой дискретизации 1000 Гц:

- Белый гауссов шум с нулевым средним и дисперсией 1.
- Розовый шум – возьмите Фурье-преобразование от белого шума, домножьте его на функцию $1/\sqrt{f}$ и возьмите обратное Фурье-преобразование, результат нормализуйте.
- красный шум – кумулятивная сумма белого шума (нормалиованная).

Постройте временные графики всех трёх сигналов. Вычислите и выведите их средние значения и дисперсии.

Вопрос: Как визуально отличаются разные виды шума? Совпадают ли вычисленные средние и дисперсии сигналов от теоретических? Если отличаются, объясните почему.

Задание 2. Вычисление автокорреляционной функции (АКФ)

Для каждого из сгенерированных шумов вычислите автокорреляционную функцию (с помощью `np.correlate(x, x, mode='full')`). Возьмите только положительные сдвиги (или сдвиги от -N+1 до N-1, но визуализируйте симметричную часть). Постройте графики нормированных АКФ (поделите на значение при нулевом сдвиге) для всех трёх шумов на одном рисунке.

Вопрос: Сравнив АКФ и временную развёртку для разных видов шума, сделайте вывод о том, как форма АКФ связана с гладкостью сигнала.

Задание 3. Оценка спектральной плотности мощности (СПМ). Периодограмма и метод Уэлча

Для каждого типа шума вычислите:

- Периодограмму с помощью `scipy.signal.periodogram`.
- Оценку СПМ методом Уэлча с помощью `scipy.signal.welch` с параметрами `nperseg=256`, `noverlap=128`, окно Ханна.

Постройте на одном графике (для каждого шума отдельно) периодограмму и оценку Уэлча в двойном логарифмическом масштабе по оси Y (используйте `plt.loglog`).

Вопрос: Как выглядят зависимости СПМ от частоты у разных видов шума в двойном логарифмическом масштабе? Как они должны выглядеть теоретически?

Задание 4. Влияние длины сегмента в методе Уэлча

Для белого шума вычислите оценку СПМ методом Уэлча с разной длиной сегмента: `nperseg = 64, 128, 256, 512`. Используйте окно Ханна, перекрытие 50%. Постройте все четыре оценки на одном графике (логарифмическая шкала по Y).

Вопрос: Как увеличение длины сегмента влияет на разрешение по частоте (ширину пиков) и гладкость оценки (дисперсию значений)?

Задание 5. Влияние дисперсии шума на его характеристики

Для каждого вида шума вычислите и выведите на один график при разных значениях дисперсии (например, 0.1, 1, 10):

1. Временную реализацию.
2. АКФ.
3. СПМ. **Вопрос:** Что меняется при изменении дисперсии в каждом случае?

Задание 6. Обнаружение гармонического сигнала на фоне шума. Спектральный анализ зашумлённой синусоиды

Сгенерируйте сигнал длительностью 2 секунды (fs=1000 Гц), состоящий из:

- синусоиды частотой 110 Гц с амплитудой 1,
- белого гауссова шума с дисперсией, обеспечивающей отношение сигнал/шум (SNR) = 10 дБ.

$$\text{SNR (дБ)} = 10\log_{10}(P_{signal}/P_{noise}).$$

1. Оцените СПМ с помощью метода Уэлча (выберите `nperseg` так, чтобы частота 110 Гц была хорошо разрешена). Определите частоту пика и сравните с истинной.
2. Повторите для других значений SNR (0 дБ, -10 дБ, -20 дБ и т. д.)

Вопрос: Как меняется видимость пика на спектре при изменении SNR? При каком SNR пик ещё можно уверенно обнаружить визуально?

Задание 7. Разрешение двух близких частот

Сгенерируйте сигнал, содержащий две синусоиды с частотами 100 и 110 Гц, равными амплитудами 1, и белый шум с дисперсией 0.1. Длительность сигнала 1 секунда, fs=1000 Гц. Используйте метод Уэлча с разными длинами сегмента: 128, 256, 512. Постройте оценки СПМ для каждого случая.

Вопрос: При какой длине сегмента удаётся разделить две частоты? Сравните с грубой теоретической оценкой частотного разрешения $\Delta f \approx f_s/N_{seg}$.

Занятие 5

Основные характеристики цифровых фильтров

Краткие сведения

1. Основные характеристики фильтров

Амплитудно-частотная характеристика (АЧХ)

- В Python: `w, H = signal.freqz(b, a, fs=fs)`
- `H` – комплексная величина, АЧХ – её модуль `np.abs(H)`

Фазо-частотная характеристика (ФЧХ)

- Вычисляется как `np.angle(H)`

Групповая задержка

- $\tau_g(f) = -\frac{d\phi(f)}{d\omega}$
- В Python: `w_gd, gd = signal.group_delay((b, a), fs=fs)`

Импульсная характеристика

- Реакция на единичный импульс $\delta[n]$
- `impulse = np.zeros(20); impulse[0] = 1`
- Вычисляется как `h = signal.lfilter(b, a, impulse)`

Переходная характеристика

- Реакция на единичный скачок $u[n]$
- `step = np.ones(50)`
- `h = signal.lfilter(b, a, step)`

2. Примеры задания фильтров по коэффициентам

Скользящее среднее (КИХ)

```
M = 5
b = np.ones(M) / M   # коэффициенты
a = [1]              # знаменатель

# Импульсная характеристика
impulse = np.zeros(20)
impulse[0] = 1
h = signal.lfilter(b, a, impulse)
```

Экспоненциальное сглаживание (БИХ)

```
alpha = 0.2
b = [alpha]
a = [1, -(1-alpha)]

y = signal.lfilter(b, a, x)
```

Передаточная функция $H(z) = \dfrac{\alpha}{1-(1-\alpha)z^{-1}}$ имеет единственный полюс $z = 1-\alpha$. Чем ближе полюс к единичной окружности (малые α), тем медленнее затухает импульсная характеристика $h[n] = \alpha(1-\alpha)^n$ и тем сильнее сглаживание. Полюса и нули находит `z, p, k = signal.tf2zpk(b, a)`.

3. Фильтр Савицкого – Голея (Savitzky–Golay)

Это КИХ-фильтр, основанный на локальной полиномиальной регрессии. Он сохраняет форму сигнала (пики, перегибы) лучше, чем скользящее среднее.

```
from scipy.signal import savgol_filter
y = savgol_filter(x, window_length=11, polyorder=2)
```

Имеется два основных параметра:

- `window_length` – длина окна усреднения (порядок фильтра)
- `polyorder` – степень используемого полинома
- Увеличение `window_length` → сильнее сглаживание, но больше искажений на краях
- Увеличение `polyorder` → лучше сохранение высоких частот, но может усиливать шум

4. Медианный фильтр

```
y = signal.medfilt(x, kernel_size=11)          # kernel_size - нечётное
y = scipy.ndimage.median_filter(x, size=11, mode='nearest')   # другая обработка
↪  краёв
```

- Заменяет отсчёт медианой значений в окне длины K.
- **Нелинейный**: не выполняется принцип суперпозиции, поэтому нет импульсной характеристики и АЧХ (импульсная характеристика формально равна нулю).
- Полностью удаляет одиночные выбросы и короткие импульсы шириной меньше $(K+1)/2$ отсчётов.
- Сохраняет резкие перепады (фронты): медиана на фронте равна значению с той стороны, где больше отсчётов окна.
- Родственные **порядковые фильтры**: максимум, минимум, k-я порядковая статистика – `signal.order_filter(x, np.ones(K), rank)`.

5. Обратить внимание

Ошибка	Исправление
Знак коэффициентов `a` для БИХ	`a = [1, -(1-alpha)]`, а не `[1, (1-alpha)]`
Нормировка коэффициентов	Сумма `b` должна быть 1 (для сохранения постоянной составляющей)
Длина `window_length` в `savgol_filter`	Должно быть нечётным
Величина `polyorder`	`polyorder` $<$ `window_length`
Длина окна в `medfilt`	`kernel_size` должно быть нечётным
Края в `medfilt`	Сигнал дополняется нулями, поэтому крайние отсчёты искажаются; используйте `scipy.ndimage.median_filter(..., mode='nearest')`

6. Ссылки на документацию

- SciPy signal processing (`https://docs.scipy.org/doc/scipy/reference/signal.html`)
- scipy.signal.freqz (`https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.freqz.html`)
- scipy.signal.group_delay (`https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.group_delay.html`)
- scipy.signal.savgol_filter (`https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.savgol_filter.html`)
- scipy.signal.medfilt (`https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.medfilt.html`)

Подготовка рабочей среды

Во всех заданиях занятия используется вспомогательная функция:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from scipy.signal import savgol_filter, savgol_coeffs, group_delay
import ipywidgets as widgets
from IPython.display import display

%matplotlib inline
plt.rcParams['figure.figsize'] = (12, 5)

# Вспомогательная функция для построения частотных характеристик
def plot_filter_response(b, a, fs=1000, title=''):
    """Строит АЧХ (линейный и дБ), ФЧХ и групповую задержку фильтра"""
    w, H = signal.freqz(b, a, fs=fs)
    mag_dB = 20*np.log10(np.abs(H) + 1e-12)
    phase = np.unwrap(np.angle(H))
    w_gd, gd = signal.group_delay((b, a), fs=fs)

    plt.figure(figsize=(12,10))
    plt.subplot(4,1,1)
    plt.semilogy(w, np.abs(H))
    plt.title(f'АЧХ (линейный масштаб) - {title}')
    plt.grid()
    plt.subplot(4,1,2)
    plt.plot(w, mag_dB)
    plt.title('АЧХ (дБ)')
    plt.ylabel('|H| (дБ)')
    plt.grid()
    plt.subplot(4,1,3)
    plt.plot(w, phase)
    plt.title('ФЧХ')
    plt.ylabel('Фаза (рад)')
    plt.grid()
    plt.subplot(4,1,4)
    plt.plot(w_gd, gd)
    plt.title('Групповая задержка')
    plt.xlabel('Частота (Гц)')
    plt.ylabel('Задержка (отсчёты)')
    plt.grid()
    plt.tight_layout()
    plt.show()
```

Часть 1. КИХ-фильтр: скользящее среднее

Скользящее среднее – простейший КИХ-фильтр. Его коэффициенты: `b_k = 1/M` для `k = 0,...,M-1`. Фильтр имеет линейную фазу.

Задание 1.1. Импульсная и переходная характеристики

Для длины окна `M = 5`:

1. Сформируйте массив коэффициентов `b` (нормированный).
2. Создайте тестовые сигналы: единичный импульс (длиной 20 отсчётов) и единичный скачок (длиной 50 отсчётов).
3. Примените фильтр с помощью `signal.lfilter(b, a, x)`, где `a = [1]`.
4. Постройте импульсную и переходную характеристики.

```
# Ваш код здесь
M = 5
# Ваш код здесь: создать массивы b и a

# Импульс
impulse = np.zeros(20)
impulse[0] = 1
# Ваш код здесь: вычислить h = signal.lfilter(...)

# Скачок
step = np.ones(50)
# Ваш код здесь: вычислить s = signal.lfilter(...)

# Построение
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.stem(..., basefmt=" ")   # Ваш код здесь: подставить h
plt.title(f'Импульсная характеристика (M={M})')
plt.grid()
plt.subplot(1,2,2)
plt.plot(...)                # Ваш код здесь: подставить s
plt.title('Переходная характеристика')
plt.grid()
plt.show()
```

Вопрос: Почему импульсная характеристика имеет конечную длину, а переходная выходит на постоянный уровень?

Задание 1.2. АЧХ и ФЧХ

Используя функцию `plot_filter_response`, постройте характеристики для скользящего среднего с длинами окон `M = 5`, `M = 10`, `M = 20`.

```
# Для M=5
b5 = np.ones(5)/5
plot_filter_response(b5, [1], fs=1000, title='Скользящее среднее M=5')

# Ваш код здесь: повторить для M=10 и M=20
```

Вопрос: Как изменение `M` влияет на крутизну среза АЧХ и на ФЧХ?

Задание 1.3. Фильтрация суммы синусоид

Сгенерируйте сигнал длительностью 1 с (fs=1000 Гц): `x = sin(2π·10·t) + 0.5·sin(2π·100·t)`. Примените скользящее среднее с `M=5` и `M=20`. Постройте временные графики и спектры (амплитудный спектр через БПФ).

```
fs = 1000
t = np.linspace(0, 1, fs)
x = np.sin(2*np.pi*10*t) + 0.5*np.sin(2*np.pi*100*t)

# Ваш код здесь: фильтрация с M=5 и M=20

# Временные графики (первые 100 отсчётов)
plt.figure(figsize=(12,4))
plt.plot(t[:100], x[:100], label='Исходный')
# Ваш код здесь: добавить графики y5, y20
plt.legend()
plt.grid()
plt.show()

# Спектры
def plot_spectrum(s, label):
    X = np.fft.fft(s)
    freq = np.fft.fftfreq(len(s), 1/fs)
    plt.plot(freq[:len(freq)//2], np.abs(X[:len(freq)//2])/(len(s)//2),
    ↪  label=label)

plt.figure(figsize=(12,4))
plot_spectrum(x, 'Исходный')
# Ваш код здесь: добавить спектры y5, y20
plt.xlim(0, 150)
plt.legend()
plt.grid()
plt.show()
```

Вопрос: Как значение `M` влияет на амплитуду 100 Гц сигнала? Объясните, почему так. Какая связь между АЧХ и результатом фильтрации?

Часть 2. БИХ-фильтр: экспоненциальное сглаживание

Фильтр первого порядка: `y[n] = α x[n] + (1-α) y[n-1]`. Коэффициенты: `b = [α]`, `a = [1, -(1-α)]`.

Задание 2.1. Реализация и характеристики

Для `α = 0.2`:

1. Постройте импульсную характеристику (50 отсчётов) и переходную характеристику.

2. Используя `plot_filter_response`, постройте АЧХ, ФЧХ и групповую задержку.
3. Запишите передаточную функцию $H(z)$ фильтра и найдите её полюс (вручную и с помощью `signal.tf2zpk(b, a)`).

```
alpha = 0.2
b = [alpha]
a = [1, -(1-alpha)]

# Импульсная характеристика (длина 50)
impulse = np.zeros(50)
impulse[0] = 1
h = signal.lfilter(b, a, impulse)
# Переходная характеристика (длина 100)
step = np.ones(100)
s = signal.lfilter(b, a, step)

plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.stem(h, basefmt=" ")
plt.title(f'Импульсная характеристика (α={alpha})')
plt.grid()
plt.subplot(1,2,2)
plt.plot(s)
plt.title('Переходная характеристика')
plt.grid()
plt.show()

# Частотные характеристики
plot_filter_response(b, a, fs=1000, title=f'Экспоненциальный α={alpha}')
```

Вопрос: Где расположен полюс передаточной функции при α `= 0.2`? Как расстояние от полюса до единичной окружности связано со скоростью затухания импульсной характеристики?

Задание 2.2. Влияние параметра α

Исследуйте фильтр при α `= 0.05, 0.2, 0.8`. Постройте на одном графике АЧХ (в дБ) и на другом – групповую задержку. Сделайте выводы.

```
alphas = [0.05, 0.2, 0.8]
# Ваш код здесь: построить АЧХ (в дБ) и групповую задержку для всех α
```

Вопрос: Как α влияет на скорость реакции фильтра на изменение сигнала и на крутизну спада АЧХ?

Задание 2.3. Фильтрация зашумлённого сигнала

Сгенерируйте сигнал: синусоида 20 Гц + белый шум (SNR $\approx$ 5 дБ), fs=500 Гц. Примените экспоненциальный фильтр с $\alpha = 0.1$ и $\alpha = 0.5$. Постройте исходный и отфильтрованные сигналы (первые 0.2 с). Сравните степень сглаживания и задержку.

```
fs = 500
t = np.linspace(0, 1, fs)
x_clean = np.sin(2*np.pi*20*t)
noise = 0.5 * np.random.randn(len(t))
x_noisy = x_clean + noise

# Ваш код здесь: фильтрация с alpha=0.1 и alpha=0.5

plt.figure(figsize=(12,4))
plt.plot(t[:100], x_noisy[:100], 'gray', alpha=0.5, label='Зашумлённый')
# Ваш код здесь: добавить графики отфильтрованных сигналов
plt.legend()
plt.grid()
plt.show()
```

Вопрос: Как на временных графиках проявляется групповая задержка, вносимая БИХ-фильтром?

Часть 3. Фильтр Савицкого – Голея (Savitzky–Golay)

Этот КИХ-фильтр основан на локальной полиномиальной регрессии. Он сохраняет форму сигнала (пики, перегибы) лучше, чем скользящее среднее.

Задание 3.1. Базовое применение и влияние параметров

Сгенерируйте синусоиду 5 Гц (fs=100 Гц, длительность 2 с) и добавьте белый шум (дисперсия 0.2). Примените `savgol_filter` с параметрами:

- (window=11, polyorder=2)
- (window=21, polyorder=2)
- (window=11, polyorder=5)

Постройте исходный чистый сигнал, зашумлённый и три сглаженные версии. Объясните влияние параметров.

```
fs = 100
t = np.linspace(0, 2, 2*fs)
x_clean = np.sin(2*np.pi*5*t)
noise = 0.2 * np.random.randn(len(t))
```

```
x_noisy = x_clean + noise

# Ваш код здесь: применить savgol_filter с разными параметрами

plt.figure(figsize=(12,6))
plt.plot(t, x_noisy, 'gray', alpha=0.4, label='Зашумлённый')
plt.plot(t, x_clean, 'k--', label='Исходный')
# Ваш код здесь: добавить графики y1, y2, y3
plt.legend()
plt.grid()
plt.show()
```

Вопрос: Как увеличение `window_length` и увеличение `polyorder` влияют на степень сглаживания и сохранение формы сигнала?

Задание 3.2. Сравнение со скользящим средним

Создайте сигнал с острым пиком (гауссов импульс с σ=3) на фоне шума. Примените скользящее среднее (window=11) и Savitzky–Golay (window=11, polyorder=2). Сравните, какой фильтр лучше сохраняет форму пика.

```
# Генерация гауссова импульса
N = 100
x_pulse = np.zeros(N)
x_pulse[N//2] = 1
sigma = 3
gaussian = np.exp(-(np.arange(N)-N//2)**2/(2*sigma**2))
x_clean_pulse = gaussian / np.max(gaussian)
noise_pulse = 0.1 * np.random.randn(N)
x_noisy_pulse = x_clean_pulse + noise_pulse

# Ваш код здесь: фильтрация (скользящее среднее и savgol)

plt.figure(figsize=(12,5))
plt.plot(x_noisy_pulse, 'gray', alpha=0.5, label='Зашумлённый')
plt.plot(x_clean_pulse, 'k--', label='Исходный импульс')
# Ваш код здесь: добавить графики отфильтрованных сигналов
plt.legend()
plt.grid()
plt.show()
```

Вопрос: Какой из фильтров лучше сохраняет пик?

Задание 3.3. Частотные характеристики

Получите коэффициенты Savitzky–Golay для `window_length=11, polyorder=2` с помощью `savgol_coeffs`. Нормируйте их так, чтобы сумма была равна 1. Постройте АЧХ этого фильтра и сравните с АЧХ скользящего среднего той же длины.

```
window_length = 11
polyorder = 2
coeffs = savgol_coeffs(window_length, polyorder)
coeffs = coeffs / np.sum(coeffs)   # нормировка

# Ваш код здесь: вычислить АЧХ через freqz для coeffs и для скользящего среднего

# Построить на одном графике (логарифмический масштаб)
```

Вопрос: Чем отличается АЧХ Savitzky–Golay от АЧХ скользящего среднего? Как это объясняет сохранение формы сигнала?

Задание 3.4. Фильтрация меандра

Сгенерируйте меандр частотой 2 Гц (fs=500 Гц) и добавьте синусоидальную помеху 50 Гц амплитудой 0.5. Примените Savitzky–Golay (window=15, polyorder=3) и скользящее среднее (window=15). Сравните, как подавляется помеха и сохраняются фронты.

```
fs = 500
t = np.linspace(0, 1, fs)
meander = signal.square(2*np.pi*2*t)
interference = 0.5 * np.sin(2*np.pi*50*t)
x_meander = meander + interference

# Ваш код здесь: фильтрация

plt.figure(figsize=(12,5))
plt.plot(t[:200], x_meander[:200], label='Исходный (меандр+помеха)')
# Ваш код здесь: добавить графики отфильтрованных сигналов
plt.legend()
plt.grid()
plt.show()
```

Вопрос: Какой фильтр лучше подходит для обработки сигналов с резкими фронтами? Почему?

Часть 4. Сравнение КИХ и БИХ фильтров

Задание 4.1. Сравнение скользящего среднего, экспоненциального фильтра и фильтра Савицкого – Голея

Для скользящего среднего (`M=10`), экспоненциального фильтра (α=`0.2`) и Savitzky–Golay (`win=11, polyorder=2`):

1. Постройте на одном графике их АЧХ (в дБ) и групповую задержку.

2. Подайте на них единичный импульс и единичный скачок, постройте реакции.
3. Сгенерируйте сигнал из прямоугольных импульсов с шумом (как в примере ниже) и примените все три фильтра. Оцените качество подавления шума и сохранения фронтов.

```
# Сигнал с прямоугольными импульсами
fs = 1000
t = np.linspace(0, 1, fs)
pulse_train = np.zeros_like(t)
pulse_train[0:100] = 1
pulse_train[250:350] = 1
pulse_train[500:600] = 1
pulse_train[750:850] = 1
noise = 0.1 * np.random.randn(len(t))
x_pulse_noisy = pulse_train + noise

# Ваш код здесь: определить коэффициенты фильтров (M=10, alpha=0.2, sg coeffs)

# Ваш код здесь: отфильтровать x_pulse_noisy каждым фильтром

# Ваш код здесь: построить графики (исходный + три отфильтрованных)
```

Вопрос: Какой фильтр обеспечивает наилучшее подавление шума? Какой лучше сохраняет фронты? Какой имеет наименьшие фазовые искажения?

Часть 5. Нелинейная фильтрация: медианный фильтр

Медианный фильтр заменяет каждый отсчёт медианой значений в окне нечётной длины K:

$$y[n] = \text{median}\{x[n-k], \ldots, x[n], \ldots, x[n+k]\}, \qquad K = 2k+1.$$

В Python: `signal.medfilt(x, kernel_size=K)`. Фильтр нелинеен: для него не выполняется принцип суперпозиции, поэтому его нельзя описать импульсной характеристикой и АЧХ.

Задание 5.1. Подавление импульсных помех

Сгенерируйте синусоиду 5 Гц (fs=500 Гц, длительность 1 с) с белым шумом ($\sigma = 0.05$). Замените 5 % случайно выбранных отсчётов значениями ± 2 (импульсная помеха типа «соль и перец»). Отфильтруйте сигнал:

- медианным фильтром с `K = 5` и `K = 11`;
- скользящим средним длины 11;
- фильтром Савицкого – Голея (window=11, polyorder=2).

Постройте графики и вычислите среднеквадратичную ошибку каждого результата относительно чистой синусоиды.

```
np.random.seed(0)
fs = 500
t = np.arange(fs) / fs
x_clean = np.sin(2*np.pi*5*t)
x_noisy = x_clean + 0.05*np.random.randn(fs)
idx = np.random.choice(fs, int(0.05*fs), replace=False)    # позиции выбросов
x_noisy[idx] = np.random.choice([-2, 2], size=len(idx))

# Ваш код здесь: медианный фильтр (K=5, 11), скользящее среднее, Савицкий - Голей

rmse = lambda y: np.sqrt(np.mean((y - x_clean)**2))
# Ваш код здесь: графики и таблица ошибок
```

Вопрос: Какой фильтр лучше справляется с импульсной помехой? Почему линейные фильтры здесь проигрывают, даже фильтр Савицкого – Голея?

Задание 5.2. Сохранение фронтов

Возьмите последовательность прямоугольных импульсов из задания 4.1 и добавьте к ней импульсную помеху: замените 2 % отсчётов значениями ± 1. Сравните медианный фильтр (`K = 11`) и скользящее среднее (`M = 11`):

1. Постройте фрагмент сигнала вокруг одного из фронтов до и после фильтрации.
2. Оцените длительность фронта после фильтрации – число отсчётов, за которое сигнал растёт от уровня 0.1 до уровня 0.9. **Вопрос:** Почему медианный фильтр не размывает фронт, а скользящее среднее размывает?

Задание 5.3. Нелинейность медианного фильтра

1. Подайте на медианный фильтр (`K = 5`) единичный импульс. Чему равна его «импульсная характеристика»?
2. Возьмите x_1 – единичный скачок (длина 50, скачок в отсчёте 25) и x_2 – одиночный выброс амплитуды 3 в отсчёте 10. Сравните `medfilt(x1 + x2)` и `medfilt(x1) + medfilt(x2)`.
3. Подайте на фильтр с `K = 5` прямоугольные импульсы шириной 1, 2, 3 и 4 отсчёта. Какие из них сохранятся после фильтрации?

```
K = 5
impulse = np.zeros(21)
impulse[10] = 1
# Ваш код здесь
```

Вопрос: Можно ли описать медианный фильтр импульсной характеристикой и АЧХ? Какие детали сигнала он удаляет, а какие сохраняет? Как минимальная ширина сохраняющегося импульса связана с длиной окна K?

Занятие 6

Синтез цифровых фильтров (КИХ и БИХ)

Краткие сведения

1. Типы цифровых фильтров

КИХ (FIR) — конечная импульсная характеристика (Finite Impulse Response)

- Всегда устойчив, проще реализация
- Можно добиться строго линейной фазы
- Требует большего порядка для резких срезов
- Разностное уравнение:

$$y[n] = \sum_{k=0}^{M} b_k x[n-k]$$

БИХ (IIR) — бесконечная импульсная характеристика (Infinite Impulse Response)

- Может быть неустойчивым
- Нелинейная фаза (искажает форму сигнала)
- Более эффективен (меньше вычислений)
- Разностное уравнение:

$$y[n] = \sum_{k=0}^{M} b_k x[n-k] - \sum_{k=1}^{N} a_k y[n-k]$$

2. Основные характеристики фильтров

АЧХ — амплитудно-частотная характеристика (показывает ослабление сигнала на разных частотах)

ФЧХ — фазо-частотная характеристика (показывает сдвиг фазы)

Групповая задержка — производная ФЧХ по частоте. Константа ⇒ линейная фаза, нет искажений формы сигнала

Параметры:

- **fc** — частота среза (граница между полосой пропускания и полосой заграждения)
- **N** — порядок фильтра или длина КИХ
- **Пульсации** — неравномерность АЧХ в полосах
- **Переходная полоса** — область между ПП и ПЗ

3. Оконный метод синтеза КИХ-фильтров

`scipy.signal.firwin(numtaps, cutoff, window='hamming', fs=None)`

Параметры:

- `numtaps` — длина фильтра (нечётное)
- `cutoff` — частота среза (по уровню −6 дБ), в тех же единицах, что и `fs`
- `window` — тип окна: 'boxcar', 'hann', 'hamming', 'blackman' и т. д.
- `fs` — частота дискретизации
- Возвращает массив коэффициентов `b`

4. Равноволновой метод

`scipy.signal.remez(numtaps, bands, desired, weight=None, fs=None)`

Параметры:

- `numtaps` — длина фильтра
- `bands` — массив границ ПП/ПЗ в тех же единицах, что и `fs` (например, `[0, 150, 250, 500]`)
- `desired` — желаемое усиление в полосах (`[1, 0]`)
- `weight` — веса полос (влияют на пульсации)
- `fs` — частота дискретизации
- Возвращает массив коэффициентов `b`

Смысл весов: Увеличение веса для полосы заграждения ⇒ глубже подавление, но больше пульсации в ПП (и наоборот).

5. БИХ-фильтры

Баттерворт

`scipy.signal.butter(N, Wn, btype='low', output='ba', fs=None)`

Параметры:

- `N` — порядок фильтра
- `Wn` — частота среза (по уровню −3дБ), в тех же единицах, что и `fs`, или нормированная от 0 до 1, если `fs=None`
- `btype` — тип фильтра (‘low’, ‘high’, ‘band’, ‘stop’)
- `output` — формат вывода (‘ba’, ‘zpk’, ‘sos’), рекомендуется ‘sos’

Чебышев тип I

`scipy.signal.cheby1(N, rp, Wn, btype='low', output='ba', fs=None)`

Параметры:

- `rp` — максимально разрешённая амплитуда пульсаций в ПП (дБ, положительное число)
- Остальные — как у `butter`

Особенности:

- Равновеликие пульсации в ПП, монотонный спад в ПЗ
- Более крутой спад, чем у Баттерворта того же порядка
- Цена — пульсации в ПП (задаются `rp`)
- Частота Wn — граница ПП, где пропускание падает ниже `-rp`

Чебышев тип II

`scipy.signal.cheby2(N, rs, Wn, btype='low', output='ba', fs=None)`

Параметры:

- `rs` — минимальное затухание, требуемое в ПЗ (дБ, положительное число)
- Остальные — как у `butter`

Особенности:

- Монотонная АЧХ в ПП, равновеликие пульсации в ПЗ
- Подавление в ПЗ не хуже rs
- Частота Wn — граница ПЗ, где затухание достигает rs

Эллиптический фильтр (Кауэра)

`scipy.signal.ellip(N, rp, rs, Wn, btype='low', output='ba', fs=None)`

Параметры:

- `rp` — пульсации в ПП (дБ)

- `rs` — затухание в ПЗ (дБ)
- Остальные — как у `butter`

Особенности:

- Пульсации и в ПП, и в ПЗ
- Самый крутой спад при заданном порядке
- Максимальная эффективность, но сильные фазовые искажения
- Частота Wn — граница ПП, где пропускание падает ниже `-rp`

Примечание: при rp→0 становится Чебышевым II, при rs→0 — Чебышевым I, при rp, rs→0 — Баттервортом

6. Анализ частотных характеристик

`scipy.signal.freqz(b, a=1, worN=512, fs=None)` `scipy.signal.freqz_sos(sos, worN=512, fs=None)`

Параметры:

- `b`, `a` — коэффициенты фильтра
- `sos` — коэффициенты фильтра в формате 'sos' (second-order sections)
- `worN` — число точек или массив частот
- `fs` — частота дискретизации

Возвращает:

- `w` — массив частот
- `h` — комплексная частотная характеристика

АЧХ в дБ: `mag_dB = 20 * np.log10(np.abs(h) + 1e-12)`

7. Групповая задержка

`scipy.signal.group_delay(system, w=512, fs=None)`

Что это? Групповая задержка показывает, на сколько отсчётов задерживаются огибающие спектральных компонент сигнала

Параметры:

- `system` — кортеж `(b, a)` (из формата 'sos' можно получить функцией `scipy.signal.sos2tf`)
- `w` — число точек/массив частот
- `fs` — частота дискретизации

Важно: Константная групповая задержка ⇒ линейная фаза, форма сигнала не искажается.

8. Фильтрация сигналов

Прямая фильтрация

`scipy.signal.lfilter(b, a, x)` `scipy.signal.sosfilt(sos, x)`

- Обычный цифровой фильтр (есть задержка, нелинейная фаза для БИХ)
- Применим для реального времени

Двухсторонняя фильтрация

`scipy.signal.filtfilt(b, a, x)` `scipy.signal.sosfiltfilt(sos, x)`

- Двунаправленная обработка (вперёд+назад)
- Нулевая фаза, сигнал не смещается по времени
- Только для офлайн-обработки

9. Полюса и нули БИХ-фильтра

`z, p, k = scipy.signal.tf2zpk(b, a)` или `z, p, k = scipy.signal.sos2zpk(sos)`

- Фильтр устойчив, если все полюса лежат внутри единичной окружности: `np.all(np.abs(p) < 1)`.
- У фильтра Баттерворта нижних частот все нули находятся в точке $z = -1$ (частота $f_s/2$), а полюса лежат на дуге внутри окружности.
- С ростом порядка полюса подходят ближе к единичной окружности вблизи частоты среза: АЧХ становится круче, а групповая задержка на этой частоте растёт.
- При высоком порядке коэффициенты `b`, `a` вычисляются с большими ошибками округления и фильтр может стать неустойчивым; формат `'sos'` (каскад звеньев второго порядка) этого избегает.

Часть 1. Синтез КИХ-фильтров методом окон

Задание 1.1. Проектирование ФНЧ с разными окнами

Спроектируйте КИХ-ФНЧ с частотой среза 200 Гц (частота дискретизации 1000 Гц), длина фильтра 51 отсчёт. Используйте окна:

- прямоугольное,
- Ханна,
- Хемминга,
- Блэкмана.

Постройте на одном графике АЧХ (в дБ) всех четырёх фильтров. Сравните:

- крутизну среза,
- уровень пульсаций в полосе пропускания и заграждения,
- ширину переходной полосы.

Вопросы: Какое окно обеспечивает наилучшее подавление в полосе заграждения? Какое – самую крутую переходную полосу? Как это связано с формой окна?

Задание 1.2. Влияние длины фильтра

Для окна Хемминга спроектируйте ФНЧ с fc=200 Гц, fs=1000 Гц, с длинами 21, 51, 101. Постройте АЧХ на одном графике.

Вопрос: Как увеличение длины влияет на крутизну среза и на уровень боковых лепестков?

Задание 1.3. Синтез ФВЧ и полосового фильтра методом окон

Используя тот же оконный метод, спроектируйте:

- ФВЧ с fc=200 Гц (длина 51, окно Хемминга);
- полосовой фильтр с полосой пропускания 200–300 Гц (длина 51, окно Хемминга).

Постройте АЧХ обоих фильтров.

Подайте на спроектированные фильтры сигнал: сумма синусоид 100, 250 и 350 Гц (амплитуды 1, 0.8, 0.6). Постройте спектры до и после фильтрации (в логарифмическом масштабе). **Вопрос:** Объясните полученные результаты.

Часть 2. Равноволновой синтез КИХ-фильтров (Паркса–МакКлеллана)

Задание 2.1. Проектирование оптимального ФНЧ

Используя `signal.remez`, спроектируйте ФНЧ с параметрами:

- частота дискретизации 1000 Гц,
- полоса пропускания 0–150 Гц,
- полоса заграждения 250–500 Гц,
- длина фильтра 21.

Постройте АЧХ (в линейном и логарифмическом масштабах) и сравните её с АЧХ КИХ-фильтра, полученного методом окон (окно Хемминга, та же длина, fc=200 Гц).

Вопросы: Какой фильтр имеет более крутой срез? Каковы пульсации в полосе пропускания?

Задание 2.2. Зависимость от ширины переходной полосы

Поменяйте верхнюю частоту полосы пропускания и нижнюю частоту полосы заграждения так, чтобы их среднее оставалось равным 200 Гц. Постройте АЧХ получившегося фильтра. Подберите ширину переходной области (разности нижней частоты полосы заграждения и верхней частоты полосы пропускания) так, чтобы уровень подавления в полосе заграждения у равноволнового фильтра совпал с оконным фильтром Хемминга.

Вопрос: При какой ширине переходной области уровень подавления в полосе заграждения у равноволнового фильтра совпал с оконным фильтром Хемминга?

Задание 2.3. Управление весами

Для фильтра из задачи 2.1 измените весовые коэффициенты: задайте `weight=[1, 10]` (увеличить вес для полосы заграждения). Постройте новую АЧХ и сравните с предыдущей.

Вопрос: Как изменилось подавление в полосе заграждения и пульсации в полосе пропускания?

Задание 2.4. Сравнение КИХ-фильтров на реальном сигнале

Сгенерируйте сигнал: смесь синусоид 50 Гц, 120 Гц, 220 Гц (амплитуды 1, 0.7, 0.3) + белый шум (дисперсия 0.1), fs=1000 Гц. Пропустите этот сигнал через:

- КИХ-ФНЧ спроектированный методом окон (окно Хемминга, длина 51, fc=150 Гц);
- КИХ-ФНЧ спроектированный методом `remez` (длина 51, полоса пропускания 0–100 Гц, полоса заграждения 200–500 Гц).

Постройте спектры исходного и отфильтрованных сигналов в линейном и логарифмическом масштабе.

Вопросы: Какой фильтр лучше подавил 220 Гц и шум? Какой лучше сохранил 120 Гц? На сколько отличаются амплитуды 120 Гц сигнала от исходных для каждого из фильтров?

Часть 3. Синтез БИХ-фильтров

Задание 3.1. Баттерворт, Чебышев, эллиптический – сравнение АЧХ

Спроектируйте ФНЧ с частотой среза 200 Гц (fs=1000 Гц) следующих типов (порядок 4):

- Баттерворт (`butter`);
- Чебышев I с пульсациями 1 дБ (`cheby1`);
- Чебышев II с затуханием 40 дБ в полосе заграждения (`cheby2`);
- Эллиптический с пульсациями 1 дБ и затуханием 40 дБ (`ellip`).

Постройте АЧХ всех фильтров на одном графике (в дБ). Сравните:

- крутизну среза,
- пульсации в полосе пропускания и заграждения.

Постройте и сравните фазовые характеристики (ФЧХ) и групповые задержки.

Вопросы: Какой фильтр даёт самый крутой спад? Какой имеет наименьшие фазовые искажения в полосе пропускания?

Задание 3.2. Влияние порядка на характеристики БИХ-фильтра

Для фильтра Баттерворта с fc=200 Гц, fs=1000 Гц возьмите порядки 2, 4, 6. Постройте АЧХ и групповую задержку.

Постройте также расположение полюсов этих фильтров на z-плоскости (`signal.tf2zpk` или `signal.sos2zpk` для формата 'sos') вместе с единичной окружностью. **Вопросы:** Как увеличение порядка влияет на крутизну среза и на групповую задержку в полосе пропускания? Как с ростом порядка меняется расположение полюсов и как это объясняет рост групповой задержки вблизи частоты среза?

Задание 3.3. Преобразование типа БИХ-фильтра (ФНЧ → ФВЧ, полосовой)

Спроектируйте Баттерворта 4-го порядка:

- ФНЧ с fc=200 Гц;
- ФВЧ с fc=200 Гц (используйте `btype='high'`);
- полосовой с полосой 200–300 Гц.

Подайте на них сигнал: сумма синусоид 100, 250, 400 Гц. Постройте спектры после фильтрации.

Вопрос: Какие составляющие подавлены, а какие пропущены в каждом случае?

Задание 3.4. Применение БИХ-фильтра к зашумлённому сигналу

Сгенерируйте сигнал: синусоида 50 Гц + белый шум с дисперсией 0.2, fs=1000 Гц, длительность 1 с. Спроектируйте эллиптический ФНЧ с fc=100 Гц (порядок 6, пульсации 1 дБ, затухание 40 дБ). Примените фильтр с помощью `lfilter` и `filtfilt`. Постройте на одном графике исходный и отфильтрованные сигналы (временные области, первые 0.2 с), а также их спектры. Сравните задержку и подавление шума.

Вопрос: Почему `filtfilt` даёт нулевой фазовый сдвиг?

Часть 4. Сравнение КИХ и БИХ фильтров одинакового порядка

Задание 4.1. Сравнение характеристик

Спроектируйте:

- КИХ-ФНЧ методом окон (окно Хемминга, длина 51, fc=200 Гц);
- БИХ-ФНЧ Баттерворта 4-го порядка (fc=200 Гц).

Постройте для них на одном графике:

- отдельно АЧХ (в дБ),
- отдельно групповую задержку.

Вопросы: Какой фильтр имеет более крутой срез? Какой имеет постоянную групповую задержку? Какой вносит меньшие фазовые искажения? Какой требует меньше вычислений? Попробуйте увеличить порядок фильтра Баттерворта. При каком порядке крутизна среза его АЧХ становится сравнимой с КИХ-фильтром? Растут или уменьшаются при этом фазовые искажения?

Задание 4.2. Применение к реальному сигналу

Сгенерируйте сигнал: короткий прямоугольный импульс длительностью 10 мс, а также высокочастотная синусоидальная помеха 400 Гц. Частоту дискретизации возьмите fs=10000 Гц.

Примените оба фильтра (КИХ и БИХ из предыдущего пункта). Постройте исходный и отфильтрованные сигналы.

Вопросы: Какой фильтр лучше сохранил форму импульсов? Какой лучше подавил помеху?

Занятие 7

Адаптивная фильтрация и частотно-временной анализ

Краткие сведения

1. Алгоритм LMS

Для КИХ-фильтра длины N на каждом отсчёте:

$$\mathbf{x}[n] = [x[n], x[n-1], \ldots, x[n-N+1]]^T, \quad y[n] = \mathbf{w}^T \mathbf{x}[n],$$
$$e[n] = d[n] - y[n], \qquad \mathbf{w} \leftarrow \mathbf{w} + \mu\, e[n]\, \mathbf{x}[n].$$

```
for n in range(N - 1, len(x)):
    x_vec = x[n - N + 1:n + 1][::-1]
    y = w @ x_vec
    e[n] = d[n] - y
    w = w + mu * e[n] * x_vec
```

- **Кривая обучения** – зависимость квадрата ошибки от номера шага в дБ. Квадрат ошибки сильно флуктуирует, поэтому его сглаживают: `np.convolve(e**2, np.ones(50)/50, mode='same')`.
- **Шаг адаптации:** малый μ – медленная сходимость, но малая установившаяся ошибка; большой – быстрая сходимость, но большая ошибка и риск расходимости.
- **Устойчивость:** для белого входного сигнала с дисперсией σ_x^2 приближённо $0 < \mu < \dfrac{2}{N\sigma_x^2}$ (точнее, $\mu < 2/\lambda_{\max}$, где $\lambda_{\max}$ – наибольшее собственное число корреляционной матрицы входа).

2. NLMS

$$\mathbf{w} \leftarrow \mathbf{w} + \frac{\mu}{\|\mathbf{x}[n]\|^2 + \varepsilon}\, e[n]\, \mathbf{x}[n], \qquad 0 < \mu < 2.$$

Шаг автоматически подстраивается под мощность входного сигнала; ε (например, 10^{-6}) защищает от деления на ноль.

3. Конфигурации адаптивного фильтра

Задача	Вход фильтра x	Желаемый сигнал d	Результат
Идентификация системы	сигнал на входе системы	сигнал на выходе системы	$\mathbf{w}$ – модель системы
Подавление шума	опорный шум	полезный сигнал + шум	ошибка e – очищенный сигнал
Эхокомпенсация	сигнал дальнего абонента	сигнал микрофона с эхом	ошибка e – речь без эха

4. Скалярный фильтр Калмана

Модель: $x_k = x_{k-1} + w_k$ (дисперсия шума процесса Q), $z_k = x_k + v_k$ (дисперсия шума измерений R).

```
x_est, P = 0.0, 1.0
for zk in z:
    P = P + Q                   # прогноз
    K = P / (P + R)             # коэффициент Калмана
    x_est = x_est + K*(zk - x_est)
    P = (1 - K) * P             # коррекция
```

В установившемся режиме $K \to K_\infty = \dfrac{-Q + \sqrt{Q^2 + 4QR}}{2R}$, и фильтр совпадает с экспоненциальным сглаживанием с $\alpha = K_\infty$.

5. STFT в SciPy: класс `ShortTimeFFT`

```
from scipy.signal import ShortTimeFFT
from scipy.signal.windows import hann

SFT = ShortTimeFFT(hann(Nw, sym=False), hop=Nw//4, fs=fs, mfft=None, scale_to=None)
X = SFT.stft(x)              # комплексные коэффициенты, форма (частоты, кадры)
S = SFT.spectrogram(x)       # |X|^2
f = SFT.f                    # частоты бинов, Гц
tt = SFT.t(len(x))           # моменты времени кадров, с
extent = SFT.extent(len(x))  # границы для plt.imshow(..., extent=extent,
↪  origin='lower', aspect='auto')
x_rec = SFT.istft(X, k1=len(x))   # обратное преобразование
```

- `hop` – шаг между окнами; перекрытие 50–75 % (`hop = Nw//2 ... Nw//4`).

- `mfft` – длина БПФ (дополнение нулями делает спектр глаже, но не улучшает разрешение).
- `scale_to='magnitude'` или `'psd'` – нормировка спектра.
- Спектрограмму показывают в дБ: `10*np.log10(S + 1e-12)`; полезно ограничить динамический диапазон (`vmin = max - 60`).
- Упрощённый вариант: `plt.specgram(x, NFFT=Nw, Fs=fs, noverlap= Nw//2)`.

Разрешение: по частоте $\Delta f \approx k\, f_s/N_w$ ($k \approx 2$ для окна Ханна), по времени $\Delta t \approx N_w/f_s$.

6. Маскирование в частотно-временной области

```
X = SFT.stft(x)
mask = np.ones(X.shape)
band = np.abs(SFT.f - f0) < df                          # строки - частоты
frames = (SFT.t(len(x)) > t1) & (SFT.t(len(x)) < t2) # столбцы - кадры
mask[np.ix_(band, frames)] = 0
y = SFT.istft(X * mask, k1=len(x))
```

7. Принцип неопределённости

$$\Delta t\, \Delta f \geq \frac{1}{4\pi},$$

где Δt и Δf – среднеквадратичные ширины распределений $|w(t)|^2$ и $|W(f)|^2$. Равенство достигается только для гауссова окна. При численном расчёте используйте симметричную частотную ось (`np.fft.fftfreq` с `np.fft.fftshift`) и большую длину БПФ.

8. Типичные ошибки

1. **Порядок отсчётов в векторе LMS:** `x[n-N+1:n+1][::-1]` – первым должен идти самый свежий отсчёт $x[n]$, иначе коэффициенты получатся в обратном порядке.
2. **Расходимость LMS:** при слишком большом шаге ошибка растёт экспоненциально и появляются `inf`/`nan`; уменьшите `mu` или используйте NLMS.
3. **Перепутаны сигналы в схеме подавления шума:** на вход фильтра подаётся опорный шум, а не сигнал микрофона.
4. **Ось частот спектрограммы:** без `extent` или `SFT.f` по осям будут номера бинов и кадров, а не герцы и секунды.
5. **Краевые эффекты STFT:** в начале и конце записи окно выходит за её пределы; яркие полосы у краёв спектрограммы – не свойство сигнала.

6. **Стерео:** аудиофайл имеет форму `(N, 2)`; перед анализом усредните каналы.

Подготовка рабочей среды

Используемые в заданиях аудиофайлы загружаются из репозитория командами:

```
# Загружаем аудиофайлы
!wget -q -O fragment.wav "https://github.com/korzhimanov/dsp-seminars/raw/refs/hea
↪ ds/main/data/fragment.wav"
!wget -q -O sample-3s.wav "https://github.com/korzhimanov/dsp-seminars/raw/refs/he
↪ ads/main/data/sample-3s.wav"
```

Часть 1. Адаптивные фильтры

Задание 1.1. Алгоритм LMS: идентификация неизвестной системы

Цель

Реализовать алгоритм LMS и с его помощью восстановить импульсную характеристику неизвестной системы.

Задание

1. «Неизвестная» система – КИХ-фильтр `h = signal.firwin(16, 0.3)`. Входной сигнал `x` – белый шум длиной 3000 отсчётов (`np.random.randn`). Желаемый сигнал – выход системы с небольшим шумом измерений: `d = signal.lfilter(h, 1, x) + 0.01*np.random.randn(len(x))`.
2. Допишите функцию `lms(x, d, N, mu)`: на каждом шаге вычисляется выход $y[n] = \mathbf{w}^T\mathbf{x}[n]$, ошибка $e[n] = d[n] - y[n]$ и обновляются коэффициенты $\mathbf{w} \leftarrow \mathbf{w} + \mu\, e[n]\, \mathbf{x}[n]$.
3. Запустите LMS с `N = 16`, `mu = 0.01`. Постройте **кривую обучения** – $10 \lg e^2[n]$, сглаженную скользящим средним по 50 отсчётам.
4. Сравните найденные коэффициенты `w` с `h` на одном графике (`plt.stem`).

```
def lms(x, d, N, mu):
    """LMS-адаптация КИХ-фильтра длины N. Возвращает коэффициенты w и сигнал ошибки
    ↪ e."""
    w = np.zeros(N)
    e = np.zeros(len(x))
    for n in range(N - 1, len(x)):
        x_vec = x[n - N + 1:n + 1][::-1]      # [x[n], x[n-1], ..., x[n-N+1]]
        # Ваш код здесь: выход фильтра, ошибка e[n], обновление w
        pass
```

```
    return w, e

def learning_curve(e, L=50):
    """Сглаженная кривая обучения в дБ."""
    return 10*np.log10(np.convolve(e**2, np.ones(L)/L, mode='same') + 1e-20)

np.random.seed(0)
h = signal.firwin(16, 0.3)
# Ваш код здесь
```

Вопросы:

- Через сколько итераций ошибка выходит на установившийся уровень? Чем определяется этот уровень?
- Насколько точно восстановлены коэффициенты неизвестной системы?

Задание 1.2. Выбор шага адаптации

1. Повторите задание 1.1 для `mu = 0.001, 0.01, 0.05, 0.1, 0.12, 0.15`. Постройте кривые обучения на одном графике.
2. Найдите экспериментально наибольшее значение `mu`, при котором алгоритм ещё сходится. Сравните его с оценкой $\mu_{\max} \approx \dfrac{2}{N\sigma_x^2}$, где σ_x^2 – дисперсия входного сигнала.
3. Увеличьте дисперсию входного сигнала в 4 раза (умножьте `x` на 2) и снова найдите порог устойчивости. **Вопросы:**

- Как шаг адаптации влияет на скорость сходимости и на установившуюся ошибку?
- Совпал ли найденный порог устойчивости с оценкой? Как он изменился при увеличении мощности входного сигнала?

Задание 1.3. NLMS при меняющейся мощности входного сигнала

1. Сформируйте вход с медленно меняющейся мощностью: `x = 3*np.random.randn(3000)*(1 + 0.9*np.sin(2*np.pi*np.arange(3000)/500))` и желаемый сигнал `d = signal.lfilter(h, 1, x)`.
2. Подберите наибольший шаг `mu`, при котором обычный LMS ещё устойчив.
3. Реализуйте NLMS: шаг нормируется на энергию вектора входных отсчётов, $\mathbf{w} \leftarrow \mathbf{w} + \dfrac{\mu}{\|\mathbf{x}[n]\|^2 + \varepsilon}\, e[n]\,\mathbf{x}[n]$, с `mu = 0.5`, `eps = 1e-6`.
4. Сравните кривые обучения LMS и NLMS.

```
def nlms(x, d, N, mu, eps=1e-6):
    """Нормированный LMS."""
```

```
    # Ваш код здесь
    pass

# Ваш код здесь
```

Вопросы:

- Почему для LMS приходится выбирать шаг по самому громкому участку сигнала?
- Почему NLMS сходится быстрее при той же устойчивости? Какой диапазон значений `mu` допустим для NLMS?

Задание 1.4. Адаптивное подавление шума

Цель

Воспроизвести схему активного шумоподавления: из сигнала микрофона вычитается оценка шума, полученная из опорного датчика.

Задание

1. Загрузите `fragment.wav`, переведите в моно (среднее каналов) и нормируйте в диапазон $[-1, 1]$. Это полезный сигнал `s`.
2. Опорный датчик регистрирует шум `ref = 0.5*np.random.randn(len(s))`. До микрофона этот шум доходит через неизвестный акустический тракт `path = signal.firwin(32, 0.2)`. Сигнал микрофона: `d = s + signal.lfilter(path, 1, ref)`.
3. Примените LMS (вход – `ref`, желаемый сигнал – `d`, `N = 32`, `mu = 0.002`). Оценкой полезного сигнала служит **сигнал ошибки** `e`.
4. Вычислите отношение сигнал/шум до и после обработки по формуле $\mathrm{SNR} = 10 \lg \dfrac{\sum s^2}{\sum (\hat{s} - s)^2}$ (после обработки – по второй половине записи, когда фильтр уже сошёлся).
5. Прослушайте `d` и `e`. Повторите эксперимент с NLMS и с длиной фильтра `N = 8`.

```
fs, s = wavfile.read('fragment.wav')
s = s.astype(float).mean(axis=1)
s /= np.max(np.abs(s))

np.random.seed(1)
# Ваш код здесь
```

Вопросы:

- Почему сигнал ошибки адаптивного фильтра оказывается очищенной речью?
- Что произойдёт, если опорный датчик будет улавливать и полезный сигнал?
- Как результат зависит от соотношения длины фильтра и длины импульсной характеристики акустического тракта?

Задание 1.5. Фильтр Калмана и экспоненциальное сглаживание

1. Смоделируйте случайное блуждание `true = np.cumsum(0.1*np.random.randn(500))` и его зашумлённые измерения `z = true + 0.5*np.random.randn(500)`.
2. Реализуйте скалярный фильтр Калмана с $F = H = 1$:
   - прогноз: $P^- = P + Q$;
   - коррекция: $K = \dfrac{P^-}{P^- + R}$, $\hat{x} \leftarrow \hat{x} + K(z - \hat{x})$, $P = (1 - K)P^-$.

   Используйте $Q = 0.01$, $R = 0.25$, начальные $\hat{x} = 0$, $P = 1$. Постройте оценку и коэффициент K как функцию номера шага.
3. Найдите установившееся значение K_∞ и сравните его с аналитическим $K_\infty = \dfrac{-Q + \sqrt{Q^2 + 4QR}}{2R}$.
4. Примените к измерениям экспоненциальное сглаживание с $\alpha = K_\infty$ (лекция 5) и сравните с оценкой Калмана. Вычислите среднеквадратичные ошибки обеих оценок и самих измерений.
5. Повторите для $Q = 0.1$ и $Q = 0.001$ при том же R.

```
np.random.seed(2)
# Ваш код здесь
```

Вопросы:

- Почему в установившемся режиме фильтр Калмана совпадает с экспоненциальным сглаживанием? В чём тогда преимущество фильтра Калмана?
- Как отношение Q/R влияет на коэффициент K и на гладкость оценки?

Часть 2. Частотно-временной анализ

Задание 2.1. Спектрограмма и выбор длины окна

1. Сформируйте сигнал длительностью 2 с при $f_s = 1000$ Гц:
   - сумма двух тонов 100 и 110 Гц на интервале $0 \le t < 1$ с;

- два коротких щелчка (одиночные отсчёты амплитуды 5) в моменты 1.50 и 1.53 с;
- линейный чирп от 200 до 400 Гц на всей длительности (`signal.chirp`).

2. Постройте спектрограммы с окном Ханна длиной `Nw = 32, 128, 512` отсчётов и шагом `Nw//8` (класс `ShortTimeFFT`, см. заметки).
3. Для каждой длины окна определите, различимы ли тоны 100 и 110 Гц и различимы ли два щелчка.
4. Объясните результат с помощью оценок разрешения: по частоте $\Delta f \approx 2f_s/N_w$ (ширина главного лепестка окна Ханна), по времени $\Delta t \approx N_w/f_s$.

```
fs = 1000
t = np.arange(0, 2, 1/fs)
# Ваш код здесь: формирование сигнала

def show_spectrogram(x, fs, Nw, ax, fmax=None):
    SFT = ShortTimeFFT(hann(Nw, sym=False), hop=max(1, Nw//8), fs=fs, mfft=max(Nw,
    ↪  1024))
    S = SFT.spectrogram(x)
    ax.imshow(10*np.log10(S + 1e-12), origin='lower', aspect='auto', cmap='magma',
              extent=SFT.extent(len(x)), vmin=10*np.log10(S.max()) - 60)
    ax.set_xlabel('Время, с'); ax.set_ylabel('Частота, Гц')
    if fmax:
        ax.set_ylim(0, fmax)
    ax.set_title(f'Окно {Nw} отсчётов')

# Ваш код здесь: три спектрограммы
```

Вопросы:

- При какой длине окна разрешаются тоны 100 и 110 Гц? При какой – два щелчка? Можно ли подобрать окно, при котором разрешается и то, и другое?
- Как выглядит чирп на спектрограммах с разной длиной окна и почему?

Задание 2.2. Принцип неопределённости

Эффективные длительность и ширина спектра окна $w[n]$ определяются через вторые моменты распределений $|w[n]|^2$ и $|W(f)|^2$:

$$\Delta t^2 = \frac{\sum_n (n-\bar{n})^2 |w[n]|^2}{\sum_n |w[n]|^2}, \qquad \Delta f^2 = \frac{\int f^2 |W(f)|^2 df}{\int |W(f)|^2 df}.$$

1. Допишите функцию `widths(w)`, вычисляющую Δt (в отсчётах) и Δf (в циклах на отсчёт). Спектр вычисляйте через БПФ с большим дополнением нулями (`nfft = 2**18`).

2. Вычислите произведение $\Delta t\,\Delta f$ для гауссовых окон длины 256 с $\sigma = 8$, 16, 32, а также для окон Ханна, треугольного и прямоугольного той же длины.
3. Сравните результаты с нижней границей $1/(4\pi) \approx 0.0796$.

```
def widths(w, nfft=2**18):
    """Эффективные длительность (в отсчётах) и ширина спектра (в циклах на отсчёт)
    ↪  окна w."""
    # Ваш код здесь
    pass

M = 256
windows = {
    'Гаусс σ=8': gaussian(M, 8), 'Гаусс σ=16': gaussian(M, 16), 'Гаусс σ=32':
    ↪  gaussian(M, 32),
    'Ханн': hann(M), 'Треугольное': triang(M), 'Прямоугольное': boxcar(M),
}
# Ваш код здесь: таблица Δt, Δf, Δt·Δf
```

Вопросы:

- Какое окно ближе всего к нижней границе? Почему для гауссова окна с большим σ произведение отклоняется от границы?
- Почему у прямоугольного окна произведение получается большим и зависит от длины БПФ?

Задание 2.3. Спектрограмма музыкального фрагмента

1. Загрузите `sample-3s.wav`, переведите в моно и нормируйте.
2. Постройте осциллограмму и спектрограммы с окнами длиной 256 и 4096 отсчётов в диапазоне частот 0–4 кГц.
3. По спектрограмме с длинным окном оцените частоту основного тона ноты, звучащей около $t = 2.3$ с: найдите частоты нескольких гармоник и расстояние между ними. Уточните результат по амплитудному спектру участка длиной 8192 отсчёта (с окном Ханна).
4. Определите, какой ноте соответствует найденная частота (ля первой октавы – 440 Гц, соседние полутона отличаются в $2^{1/12}$ раз).

```
fs_a, audio = wavfile.read('sample-3s.wav')
# Ваш код здесь
```

Вопросы:

- Что лучше видно на спектрограмме с коротким окном, а что – с длинным?
- Какова частота основного тона и какая это нота?

Задание 2.4. Обработка в частотно-временной области

1. Добавьте к музыкальному фрагменту из задания 2.3 помеху – тон 3 кГц амплитудой 0.2, звучащий только на интервале от 1 до 2 с.
2. Вычислите STFT (`SFT.stft`) с окном Ханна длиной 1024 и шагом 256. Убедитесь, что обратное преобразование (`SFT.istft(X, k1=len(x))`) без изменений восстанавливает сигнал с точностью до ошибок округления.
3. Обнулите коэффициенты STFT в полосе 3000 ± 50 Гц на интервале 0.9–2.1 с и выполните обратное преобразование.
4. Сравните спектрограммы и прослушайте сигналы до и после обработки. Вычислите отношение сигнал/помеха до и после.
5. Попробуйте вместо этого убрать помеху обычным режекторным фильтром (`signal.iirnotch`). Чем отличается результат? **Вопросы:**

- Почему маску удалось ограничить и по частоте, и по времени? Что произойдёт с музыкой, если обнулить ту же полосу на всей длительности?
- Чем обработка в частотно-временной области лучше обычного режекторного фильтра для помехи, которая появляется на короткое время?

Занятие 8

Вейвлет-преобразования

Краткие сведения

1. Библиотека PyWavelets

```
!pip install -q PyWavelets      # в Colab обычно уже установлена
import pywt
pywt.wavelist(kind='continuous')  # вейвлеты для CWT
pywt.wavelist(kind='discrete')    # вейвлеты для DWT
```

Имена вейвлетов:

- `'haar'`, `'dbN'` (Добеши, N нулевых моментов, фильтр длины 2N), `'symN'` (симлеты), `'coifN'` (койфлеты), `'biorNr.Nd'` (биортогональные) – для DWT;
- `'mexh'` (мексиканская шляпа), `'morl'` (вещественный Морле), `'gausN'` (производные гауссиана), `'cmorB-C'` (комплексный Морле с шириной B и центральной частотой C) – для CWT.

2. Непрерывное преобразование (CWT)

```
fs = 1000
freqs = np.geomspace(5, 400, 150)                       # нужные частоты, Гц
scales = pywt.central_frequency('cmor1.5-1.0') * fs / freqs
W, f = pywt.cwt(x, scales, 'cmor1.5-1.0', sampling_period=1/fs, method='fft')
plt.pcolormesh(t, f, np.abs(W), shading='auto')
plt.yscale('log')
```

- Связь масштаба и частоты: $f = \dfrac{f_c}{a\,\Delta t}$, где f_c – центральная частота вейвлета (`pywt.central_frequency`), $\Delta t = 1/f_s$.
- `W` имеет форму `(число масштабов, длина сигнала)`; для комплексного вейвлета значения комплексные, для анализа берут `np.abs(W)`.
- `method='fft'` значительно быстрее для длинных сигналов и большого числа масштабов.

- **Конус влияния** для `cmorB-C`: на масштабе a краевые эффекты распространяются на $a\sqrt{B/2}$ отсчётов от краёв записи.
- Ширина B: больше B – больше осцилляций в вейвлете, лучше разрешение по частоте и хуже по времени.

3. Дискретное преобразование (DWT)

```
cA, cD = pywt.dwt(x, 'db4', mode='symmetric')       # один уровень
coeffs = pywt.wavedec(x, 'db4', level=5)             # [cA5, cD5, cD4, cD3, cD2, cD1]
x_rec = pywt.waverec(coeffs, 'db4')                  # может оказаться на 1 отсчёт
↪  длиннее x
J = pywt.dwt_max_level(len(x), 'db4')                # максимальное число уровней
```

- **Порядок коэффициентов** в `wavedec`: сначала аппроксимация самого грубого уровня, затем детали от грубых к мелким. Деталь d_j (первый уровень – самый мелкий) соответствует полосе $[f_s/2^{j+1},\ f_s/2^j]$.
- **Режим продолжения** `mode`: `'symmetric'` (по умолчанию), `'periodization'` (длина коэффициентов ровно вдвое меньше на каждом уровне – удобно для сжатия), `'zero'`, `'smooth'` и др.
- **Фильтры:** `w = pywt.Wavelet('db4')`; `w.dec_lo`, `w.dec_hi` – анализ, `w.rec_lo`, `w.rec_hi` – синтез; `w.wavefun(level=8)` – значения φ и ψ.
- Ручная реализация одного уровня (совпадает с `mode='zero'`):

```
a = np.convolve(x, w.dec_lo)[1::2]
d = np.convolve(x, w.dec_hi)[1::2]
```

4. Вклад отдельного уровня

```
def component(coeffs, i, wavelet):
    c = [np.zeros_like(ci) for ci in coeffs]
    c[i] = coeffs[i]
    return pywt.waverec(c, wavelet)
```

5. Пороговая обработка

```
sigma = np.median(np.abs(coeffs[-1])) / 0.6745       # оценка шума по деталям 1-го
↪  уровня
lam = sigma * np.sqrt(2 * np.log(len(x)))            # универсальный порог
coeffs_t = [coeffs[0]] + [pywt.threshold(d, lam, mode='soft') for d in coeffs[1:]]
```

- Жёсткий порог (`'hard'`) обнуляет малые коэффициенты и не меняет большие; мягкий (`'soft'`) дополнительно уменьшает большие на λ – результат глаже, но амплитуды занижены.
- Аппроксимацию обычно не обрабатывают.

6. Сжатие: работа со всеми коэффициентами сразу

```
arr, slices = pywt.coeffs_to_array(coeffs)           # все коэффициенты в одном
↪  массиве
k = int(p * arr.size)
thr = np.sort(np.abs(arr).ravel())[-k]
arr_c = np.where(np.abs(arr) >= thr, arr, 0)
coeffs_c = pywt.array_to_coeffs(arr_c, slices, output_format='wavedec')   #
↪  'wavedec2' для 2D
```

Для изображений: `pywt.wavedec2(img, 'bior4.4', level=3)`, `pywt.waverec2`. Каждый уровень деталей – кортеж из трёх субполос (горизонтальные, вертикальные, диагональные детали). Аппроксимацию `arr[slices[0]]` при сжатии лучше сохранять целиком.

PSNR для 8-битного изображения: `10*np.log10(255**2 / np.mean((img_rec - img)**2))`.

7. Нулевые моменты

Вейвлет с p нулевыми моментами даёт нулевые детали для многочленов степени меньше p. У Добеши dbN $p = N$: `pywt.Wavelet('db3').vanishing_moments_psi`.

8. Типичные ошибки

1. **Перепутан порядок уровней:** `coeffs[-1]` – самые мелкие детали d_1, а не самые крупные.
2. **Длина после `waverec`:** при нечётной длине или режиме, отличном от `'periodization'`, восстановленный сигнал может быть длиннее исходного – обрежьте `x_rec[:len(x)]`.
3. **Масштабы вместо частот:** `pywt.cwt` принимает масштабы; частоты нужно пересчитать через центральную частоту вейвлета.
4. **Линейная ось частот на скалограмме:** при геометрической сетке масштабов используйте логарифмическую ось.
5. **Вещественный вейвлет для частотного анализа:** у `'morl'` и `'mexh'` модуль коэффициентов осциллирует во времени; для оценки огибающей используйте `'cmor'`.
6. **Число уровней:** уровней больше `pywt.dwt_max_level` вызывают предупреждение и сильные краевые эффекты.

Часть 1. Непрерывное вейвлет-преобразование

Задание 1.1. Масштаб и частота

1. Выведите центральные частоты (`pywt.central_frequency`) вейвлетов `'mexh'`, `'morl'`, `'cmor1.5-1.0'`, `'gaus1'`.
2. Сформируйте тон 50 Гц длительностью 1 с при $f_s = 1000$ Гц. Задайте 100 частот анализа в геометрической прогрессии от 10 до 200 Гц (`np.geomspace`) и пересчитайте их в масштабы по формуле $a = f_c f_s / f$.
3. Вычислите CWT с вейвлетом `'cmor1.5-1.0'` (`pywt.cwt(x, scales, wavelet, sampling_period=1/fs)`). Постройте зависимость $|W|$, усреднённого по центральной половине записи, от частоты. Совпадает ли максимум с 50 Гц?
4. Повторите для вейвлетов `'mexh'` и `'gaus1'`. Постройте для них также зависимость $|W(a, b)|$ от времени b на масштабе, соответствующем 50 Гц.

```
fs = 1000
t = np.arange(0, 1, 1/fs)
x = np.sin(2*np.pi*50*t)
# Ваш код здесь
```

Вопросы:

- Насколько точно максимум $|W|$ указывает частоту тона для каждого вейвлета?
- Почему для вещественных вейвлетов $|W(a, b)|$ осциллирует во времени, а для комплексного вейвлета Морле – нет?
- Почему для некоторых вейвлетов максимум заметно смещён относительно 50 Гц? (Подсказка: центральная частота – лишь приближённая характеристика широкополосного вейвлета; сравните спектры вейвлетов.)

Задание 1.2. Скалограмма нестационарного сигнала

1. Сформируйте сигнал длительностью 2 с при $f_s = 1000$ Гц: тон 20 Гц амплитудой 0.6 на всей длительности; чирп от 40 до 250 Гц на интервале 0.4–1.6 с (`signal.chirp`); два щелчка (гауссовы импульсы шириной 2 мс) в моменты 0.25 и 1.8 с.
2. Постройте скалограмму $|W(a, b)|$ (вейвлет `'cmor1.5-1.0'`, 150 частот от 4 до 400 Гц) с логарифмической осью частот (`plt.pcolormesh` и `plt.yscale('log')`).
3. Нанесите на скалограмму **конус влияния**: для вейвлета `cmorB-C` на масштабе a краевые эффекты распространяются на расстояние $a\sqrt{B/2}$ отсчётов от краёв записи.

4. Постройте спектрограмму того же сигнала с окном 128 отсчётов и сравните её со скалограммой.

```
fs = 1000
t = np.arange(0, 2, 1/fs)
# Ваш код здесь
```

Вопросы:

- Как выглядят на скалограмме тон, чирп и щелчки? Почему щелчки сужаются к высоким частотам?
- Какие детали лучше видны на скалограмме, а какие – на спектрограмме?

Задание 1.3. Параметр ширины вейвлета Морле

1. Сформируйте сигнал длительностью 2 с при $f_s = 1000$ Гц: сумма тонов 40 и 50 Гц на интервале 0–1 с и два щелчка (одиночные отсчёты амплитуды 20) в моменты 1.50 и 1.52 с.
2. Постройте скалограммы с вейвлетами `'cmorB-1.0'` при $B = 0.5,\ 1.5,\ 5,\ 20$ (частоты 20–150 Гц).
3. Для каждого B определите, различимы ли тоны 40 и 50 Гц и различимы ли два щелчка на частотах около 100 Гц. Для анализа щелчков постройте увеличенный фрагмент скалограммы на интервале 1.45–1.57 с. **Вопросы:**

- При каких B разрешаются тоны? При каких – щелчки? Существует ли B, при котором разрешается и то, и другое?
- Как этот результат связан с принципом неопределённости?

Часть 2. Дискретное вейвлет-преобразование

Задание 2.1. Преобразование Хаара вручную

1. Допишите функцию `haar_step(a)`, выполняющую один уровень ортонормированного преобразования Хаара: $s_k = (a_{2k} + a_{2k+1})/\sqrt{2},\quad d_k = (a_{2k} - a_{2k+1})/\sqrt{2}$.
2. Выполните полное разложение сигнала $x = [4,\ 2,\ 5,\ 5,\ 1,\ 3,\ 6,\ 6]$ (повторяя шаг для аппроксимации, пока не останется один коэффициент). Сравните с `pywt.wavedec(x, 'haar')`.
3. Напишите обратное преобразование и убедитесь, что сигнал восстанавливается точно.
4. Проверьте равенство Парсеваля: энергия сигнала равна энергии коэффициентов.

```
def haar_step(a):
    """Один уровень преобразования Хаара: возвращает (аппроксимация, детали)."""
    # Ваш код здесь
    pass

def haar_inverse_step(s, d):
    """Обратный шаг: восстанавливает сигнал по аппроксимации и деталям."""
    # Ваш код здесь
    pass

x = np.array([4, 2, 5, 5, 1, 3, 6, 6], dtype=float)
# Ваш код здесь
```

Вопрос: Почему многие коэффициенты оказались нулевыми? Какие свойства сигнала они отражают?

Задание 2.2. Один уровень алгоритма Малла

1. Возьмите фильтры вейвлета Добеши db4: `w = pywt.Wavelet('db4')`, фильтры анализа `w.dec_lo`, `w.dec_hi`, синтеза – `w.rec_lo`, `w.rec_hi`.
2. Реализуйте один уровень разложения: свёртка с фильтром анализа (`np.convolve`) и прореживание – взять отсчёты с нечётными индексами (`[1::2]`). Сравните результат с `pywt.dwt(x, 'db4', mode='zero')` для случайного сигнала длиной 64.
3. Реализуйте синтез: вставка нулей между коэффициентами, свёртка с фильтрами синтеза, сложение ветвей. Восстановленный сигнал начинается с отсчёта номер $L-2$, где L – длина фильтра.
4. Постройте АЧХ фильтров `dec_lo` и `dec_hi` и проверьте, что $|H|^2+|G|^2 = 2$.

```
w = pywt.Wavelet('db4')
np.random.seed(0)
x = np.random.randn(64)
# Ваш код здесь
```

Вопросы:

- Почему после прореживания информация не теряется, хотя каждая ветвь содержит вдвое меньше отсчётов?
- Почему сложность полного многоуровневого разложения пропорциональна N?

Задание 2.3. Многоуровневое разложение и частотные полосы

1. Сформируйте сигнал $x(t) = \sin(2\pi \cdot 10t) + \sin(2\pi \cdot 60t) + \sin(2\pi \cdot 200t)$ длительностью 2.048 с при $f_s = 1000$ Гц.

2. Выполните разложение `pywt.wavedec(x, 'db8', level=6)`. Учтите порядок коэффициентов в списке: $[a_6, d_6, d_5, \ldots, d_1]$.
3. Составьте таблицу: уровень, полоса частот (d_j соответствует $[f_s/2^{j+1},\ f_s/2^j]$, a_J – $[0,\ f_s/2^{J+1}]$), доля энергии сигнала в коэффициентах уровня.
4. Восстановите вклад каждого уровня в отдельности (обнулив остальные коэффициенты и применив `pywt.waverec`) и постройте графики.

```
fs = 1000
t = np.arange(0, 2.048, 1/fs)
# Ваш код здесь
```

Вопросы:

- В коэффициентах каких уровней оказался каждый из тонов?
- Почему энергия тона 60 Гц распределилась между двумя уровнями?

Задание 2.4. Нулевые моменты

1. Сформируйте кусочно-полиномиальный сигнал из 512 отсчётов на отрезке $[0, 1)$: прямая $2t$ при $t < 0.5$, парабола $1.5 - 4(t - 0.5)^2$ при $t \geq 0.5$ и скачок на $+0.5$ при $t \geq 0.75$.
2. Выполните один уровень DWT (`pywt.dwt(x, wavelet, mode='periodization')`) для вейвлетов `haar`, `db2`, `db3`, `db4`. Постройте детали d_1.
3. Для каждого вейвлета подсчитайте число деталей с модулем больше 10^{-6}. Число нулевых моментов вейвлета можно узнать из `pywt.Wavelet(name).vanishing_moments_psi`.

```
N = 512
n = np.arange(N) / N
# Ваш код здесь
```

Вопросы:

- Как число ненулевых деталей связано с числом нулевых моментов вейвлета?
- Почему ненулевые детали появляются у краёв записи?

Часть 3. Применения

Задание 3.1. Подавление шума

1. Возьмите тестовые сигналы `pywt.data.demo_signal('Blocks', 1024)` и `pywt.data.demo_signal('Doppler', 1024)`, нормируйте их на стандартное отклонение и добавьте белый шум с $\sigma = 0.25$.

2. Реализуйте вейвлет-денойзинг: разложение `wavedec` (5 уровней, `mode='periodization'`), оценка шума $\hat{\sigma} = \mathrm{median}(|d_1|)/0.6745$, универсальный порог $\lambda = \hat{\sigma}\sqrt{2\ln N}$, пороговая обработка всех деталей (`pywt.threshold`), обратное преобразование.
3. Сравните мягкий и жёсткий пороги и вейвлеты `haar`, `db4`, `sym8`. Сведите среднеквадратичные ошибки в таблицу.
4. Сравните лучший результат с фильтром Баттерворта 4-го порядка (`signal.butter` + `signal.filtfilt`), подобрав частоту среза так, чтобы ошибка была минимальной.

```
np.random.seed(3)
N = 1024
# Ваш код здесь
```

Вопросы:

- Какой вейвлет и какой порог лучше подходят для каждого сигнала? Почему?
- Насколько близка оценка $\hat{\sigma}$ к истинному уровню шума?
- Почему вейвлет-денойзинг сохраняет фронты лучше линейного фильтра?

Задание 3.2. Сжатие сигнала

1. Возьмите сигнал `pywt.data.demo_signal('Piece-Regular', 1024)`.
2. Выполните полное разложение (`wavedec`, вейвлет `db4`, `mode='periodization'`) и сохраните долю p наибольших по модулю коэффициентов, обнулив остальные (удобно использовать `pywt.coeffs_to_array` и `pywt.array_to_coeffs`). Восстановите сигнал.
3. Сделайте то же с коэффициентами ДКП (`dct(x, norm='ortho')`, `idct`).
4. Постройте зависимость относительной ошибки $\|\hat{x} - x\|/\|x\|$ от p в диапазоне 1–20 % для обоих преобразований (логарифмическая шкала по оси ошибки).
5. Повторите для гладкого сигнала `'HeaviSine'`. **Вопросы:**

- Для какого сигнала вейвлет-преобразование выигрывает у ДКП, а для какого – нет? Почему?

Задание 3.3. Сжатие изображения

1. Загрузите изображение `pywt.data.camera()` (512×512, оттенки серого).
2. Выполните двумерное разложение `pywt.wavedec2(img, wavelet, level=3, mode='periodization')`. Изобразите коэффициенты (`pywt.coeffs_to_array`), нормировав каждую субполосу.

3. Сохраните аппроксимацию целиком, а из деталей – наибольшие по модулю коэффициенты так, чтобы всего осталось 1, 2, 5 и 10 % коэффициентов. Восстановите изображение и вычислите PSNR: $10 \lg \dfrac{255^2}{(\hat{I} - I)^2}$.
4. Сравните вейвлеты `haar`, `db4` и `bior4.4` (последний близок к CDF 9/7 из JPEG2000).

```
img = pywt.data.camera().astype(float)
# Ваш код здесь
```

Вопросы:

- Какие артефакты появляются при сильном сжатии у разных вейвлетов?
- Какой вейвлет даёт наибольший PSNR? Почему в JPEG2000 используют биортогональные вейвлеты?

Задание 3.4. Обработка ЭКГ

1. Загрузите запись ЭКГ `pywt.data.ecg()` (частота дискретизации 360 Гц), вычтите среднее и нормируйте на максимум модуля.
2. Добавьте типичные помехи: дрейф изолинии $0.8 \sin(2\pi \cdot 0.3t + 1)$, сетевую наводку $0.15 \sin(2\pi \cdot 50t)$ и белый шум с $\sigma = 0.05$.
3. Попробуйте найти R-зубцы в зашумлённом сигнале напрямую (`signal.find_peaks` с порогом по высоте и минимальным расстоянием 0.25 с).
4. Выполните разложение `wavedec(y, 'db4', level=7)`. Составьте таблицу частотных полос уровней.
5. **Удаление дрейфа:** обнулите аппроксимацию и восстановите сигнал. Сравните с исходной ЭКГ.
6. **Поиск R-зубцов:** восстановите сигнал только по деталям уровней, в полосы которых попадает энергия QRS-комплекса (примерно 5–45 Гц), и найдите пики квадрата этого сигнала. Вычислите частоту сердечных сокращений.

```
ecg = pywt.data.ecg().astype(float)
fs = 360
t = np.arange(len(ecg)) / fs
np.random.seed(4)
# Ваш код здесь
```

Вопросы:

- Почему прямой поиск пиков в зашумлённом сигнале не работает?
- Какие уровни разложения вы использовали для поиска R-зубцов и почему? Какова частота сердечных сокращений?

Список литературы

Общие вопросы цифровой обработки сигналов

[1.1] Оппенгейм А., Шафер Р. Цифровая обработка сигналов. — М.: Техносфера, 2006.

[1.2] Лайонс Р. Цифровая обработка сигналов. 2-е изд. — М.: Бином-Пресс, 2006.

[1.3] Сергиенко А.Б. Цифровая обработка сигналов. 3-е изд. — СПб.: БХВ-Петербург, 2011.

[1.4] Smith S. W. The Scientist and Engineer's Guide to Digital Signal Processing. — San Diego: California Technical Publishing, 1997.

Спектральный анализ и синтез фильтров

[2.1] Welch P. D. The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms // IEEE Transactions on Audio and Electroacoustics. — 1967. — V. 15. — P. 70–73.

[2.2] Savitzky A., Golay M. J. E. Smoothing and Differentiation of Data by Simplified Least Squares Procedures // Analytical Chemistry. — 1964. — V. 36. — P. 1627–1639.

[2.3] Parks T. W., McClellan J. H. Chebyshev Approximation for Nonrecursive Digital Filters with Linear Phase // IEEE Transactions on Circuit Theory. — 1972. — V. 19. — P. 189–194.

Адаптивная фильтрация

[3.1] Haykin S. Adaptive Filter Theory. 5th ed. — Upper Saddle River: Pearson, 2014.

[3.2] Kalman R. E. A New Approach to Linear Filtering and Prediction Problems // Journal of Basic Engineering. — 1960. — V. 82. — P. 35–45.

Вейвлет-преобразования

[4.1] Добеши И. Десять лекций по вейвлетам. — Ижевск: НИЦ «Регулярная и хаотическая динамика», 2001.

[4.2] Mallat S. A Wavelet Tour of Signal Processing: The Sparse Way. 3rd ed. — Burlington: Academic Press, 2009.

[4.3] Donoho D. L., Johnstone I. M. Ideal spatial adaptation by wavelet shrinkage // Biometrika. — 1994. — V. 81. — P. 425–455.

Программные средства

[5.1] Virtanen P. et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python // Nature Methods. — 2020. — V. 17. — P. 261–272.

[5.2] Lee G. R., Gommers R., Waselewski F., Wohlfahrt K., O'Leary A. PyWavelets: A Python package for wavelet analysis // Journal of Open Source Software. — 2019. — V. 4. — Art. no. 1237.

Артем Владимирович **Коржиманов**

ЦИФРОВАЯ ОБРАБОТКА СИГНАЛОВ ПРАКТИКУМ

Учебно-методическое пособие

Компьютерная верстка – А.В. Коржиманов

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского».
603950, Нижний Новгород, пр. Гагарина, 23.